

TF7xxx | TwinCAT Vision Quickstart



Contents

1. Introduction	5
2. Overview	6
2.1 Product introduction	6
2.2 System components	7
2.3 GigE Vision	9
2.4 GenICam	10
2.5 Software Architecture in TwinCAT Runtime	10
2.6 General functions	11
2.6.1 Object detection	12
2.6.2 Measurement	12
2.6.3 Identification and code quality inspection	13
2.6.4 Monitoring	13
2.6.5 OCR	14
2.6.6 Anomaly detection	14
3. Product overview	16
3.1 Software	16
3.1.1 Overview	16
3.1.2 Licensing	18
4. TwinCAT Vision in the XAE	19
4.1 Camera	19
4.2 File Source Control	20
5. Prerequisites for the exercises	23
5.1 Cloud environment	23
5.2 Local engineering station	23
6. Project sample - Basic Vision Project	24
6.1 Objective	24
6.2 Creating a TwinCAT XAE solution for a vision application	24
6.3 Vision node – File source configuration	27
6.4 Base PLC project for vision applications	28
6.5 Conclusion	31
7. Project sample - Image analysis: Colors and contours	32
7.1 Objective	32
7.2 Initial setup (see base exercise for details)	32
7.3 Implementation	35
8. Project sample - TwinCAT Vision and TwinCAT HMI	42
8.1 Introduction	42

8.2 Project Sample43

8.2.1 Objective43

8.2.2 Commissioning43

8.2.3 Implementation44

1. Introduction

This document is made with the sole purpose of giving a rough feature overview and quick start of the capabilities of the TF7xxx TwinCAT Vision PLC libraries and their implementation in TwinCAT XAE.

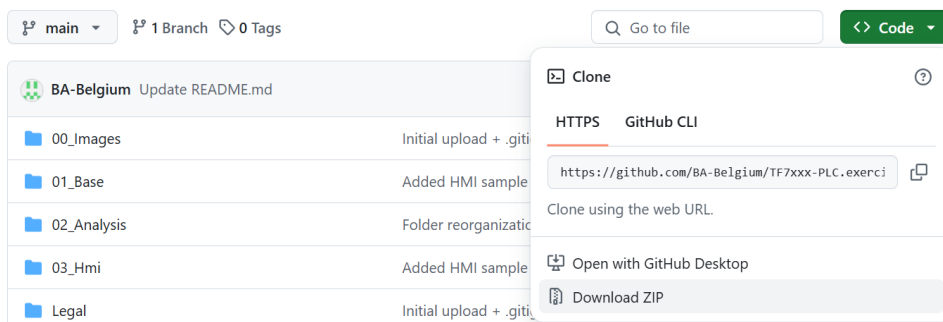
This quick start does not replace any official documentation or training. Please refer to the local Beckhoff subsidiary for the complete training session before even considering yourself a TwinCAT and/or a TwinCAT Vision specialist.

For this quickstart TwinCAT 4026 was used, but the exercises contained can be implemented in TwinCAT 4024 in the exact same way.

The template sources, complete projects and images for this quickstart can be found on the Beckhoff Belgium github account.

<https://github.com/BA-Belgium/TF7xxx-PLC.exercises.quickstart.plc>

For the exercises: open the link, download the source code (download zip) and extract the zip in the desired location on your development station.

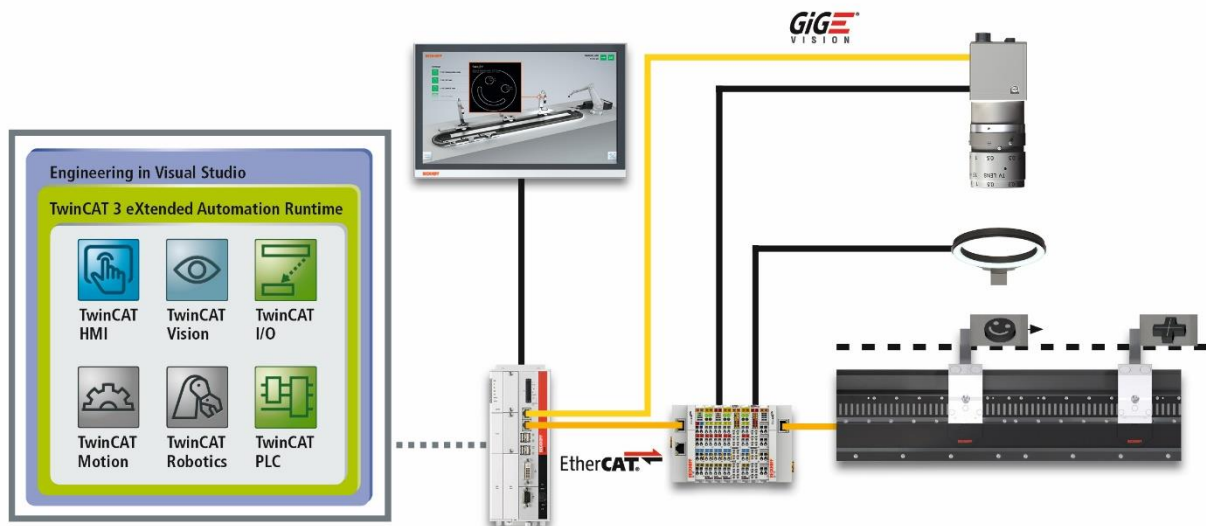


Further information can be found here:

- Infosys documentation
<https://infosys.beckhoff.com/content/1033/tcinfosys3/5860898315.html?id=2433051221277546222>
- Documentation download
https://download.beckhoff.com/download/document/automation/twincat3/TF7000-TF7800_TC3_Vision_EN.pdf
- A very basic PLC template as a starting point for TcVision applications
<https://github.com/BA-Belgium/TF7xxx-PLC.sample.template.plc>
- The official Beckhoff samples for a variety of applications
https://github.com/Beckhoff/TF7xxx_Samples
- The official Beckhoff picture library, that you can use for further exercises
https://download.beckhoff.com/download/software/GitHub/TF7xxx/TF7xxx_Sample_Images.zip

2. Overview

2.1 Product introduction



TwinCAT Vision is a TwinCAT 3 function for industrial image processing. Tasks such as the detection, identification or measurement of objects can be handled directly in the PLC in real-time. By integrating image processing into the TwinCAT platform, highly synchronized control applications and extremely short response times can be achieved. Application development is aided by the large number of existing TwinCAT Engineering tools.

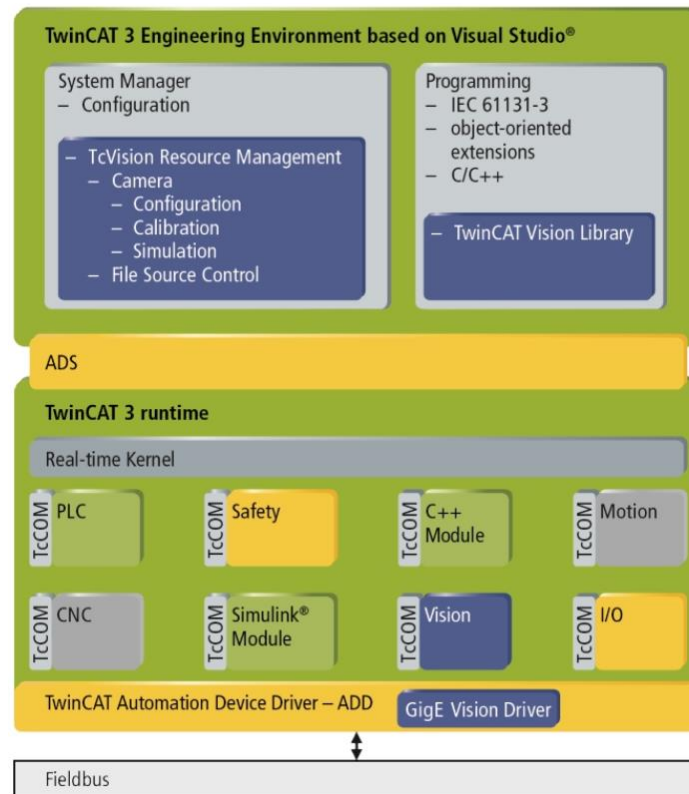
Image processing sequences are programmed in the IEC61131-3 languages using the Tc3_Vision library directly in the PLC code. Intermediate results can be checked directly using an image display tool.

TwinCAT Vision is real-time capable because the image processing algorithms are executed synchronously with the controller directly in the TwinCAT 3 runtime environment. In addition, suitable algorithms can run automatically in parallel on several cores.

A large number of industrial cameras can be connected via the GigE Vision interface. The familiar TwinCAT development environment is used for configuring and calibrating cameras, and for system setup and debugging.

2.2 System components

Like other TwinCAT components, TwinCAT Vision is integrated into the Visual Studio-based TwinCAT development environment and the real-time capable TwinCAT runtime environment. The components of the TwinCAT Vision system are explained below.



Configuration Assistants

Inside the XAE (TwinCAT 3 Engineering Environment) TwinCAT Vision integrates with the system manager with the TcVision Resource Management. The TcVision Resource Management consists of graphical configuration assistants to facilitate the configuration, calibration and simulation of industrial GigE Vision Camera's. Images can also be fed from the file system into the PLC runtime via the File Source assistant.

PLC Library

TwinCAT Vision provides a PLC library with a large number of algorithms for programming image processing sequences. It also contains function blocks that serve as interfaces for cameras and the file system. The API reference contains detailed descriptions of all library elements. The algorithms behind the PLC library are written in C++.

GigE Vision driver

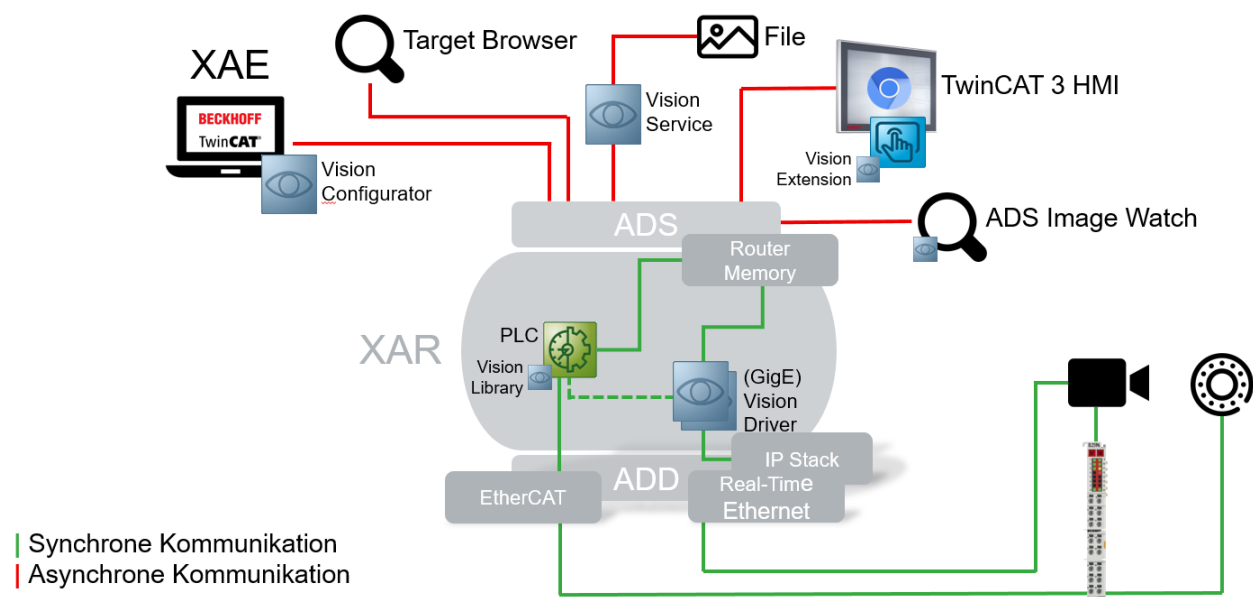
TwinCAT Vision includes a real-time capable GigE Vision driver. It is used in the development environment for camera configuration and in the TwinCAT runtime. It communicates with industrial cameras via UDP/IP, based on the GigE Vision standard. This standard is used to find and configure cameras in the network and to receive image data from the cameras.

TwinCAT Vision Service

For some functionalities, TwinCAT Vision has to communicate with components outside the real-time at runtime. For example, a File Source Control loads images from the file system. The TwinCAT Vision Service is a centralized, ADS-based communication interface between the TwinCAT runtime and external Vision components. The Vision Service enables the TwinCAT Vision API function blocks to indirectly access the hard disk from the real-time environment, e.g. to load or save images.

TwinCAT Vision HMI Extension

The TwinCAT HMI Vision Extension is an extension for the TwinCAT HMI that enables transfer of images from the PLC to the TwinCAT HMI. Images from the PLC can be displayed directly in the web-based HMI with tcHmiImageControl.



2.3 GigE Vision



TwinCAT Vision or the TC3 GigE Vision Connector product is certified to GigE Vision (Version 2.0.3). TwinCAT Vision can thus be used with all GigE Vision-compatible cameras.

GigE Vision is an interface standard for industrial image acquisition. It facilitates configuration and operation of industrial cameras. GigE stands for Gigabit Ethernet. By using the Gigabit Ethernet communication protocol, GigE Vision benefits from the following features:

- The default transmission rate is 1 Gigabit/s (extendable with 2.5 GigE, 5 GigE and 10 GigE).
- Cable lengths up to 100 meters without the need for amplification.
- Use of standard hardware and software interfaces for Gigabit Ethernet.

The GigE Vision standard defines communication with compatible devices via UDP/IP and consists of the following four elements:

- The GigE Vision Control Protocol (GVCP) defines how GigE Vision devices are to be addressed. It specifies the data channels and mechanisms for the transfer of images and configuration data on register basis between PC and camera.
- The GigE Vision Stream Protocol (GVSP) specifies the different data types and transmission methods used to transfer images from a camera to a PC (=streaming). A Packet Resend option is also available to fix transmission errors.
- The GigE Device Discovery Mechanism makes it possible to find cameras in the network using search queries.
- An XML file contains the GenAPI description that defines all public functions of the camera. This description is based on the GenICam standard.

2.4 GenICam



GenICam stands for **Generic Interface for Cameras**. The GenICam standard describes the camera's properties in an XML file and therefore enables easy access to the camera's settings. The format of the description depends on the camera's type and manufacturer.

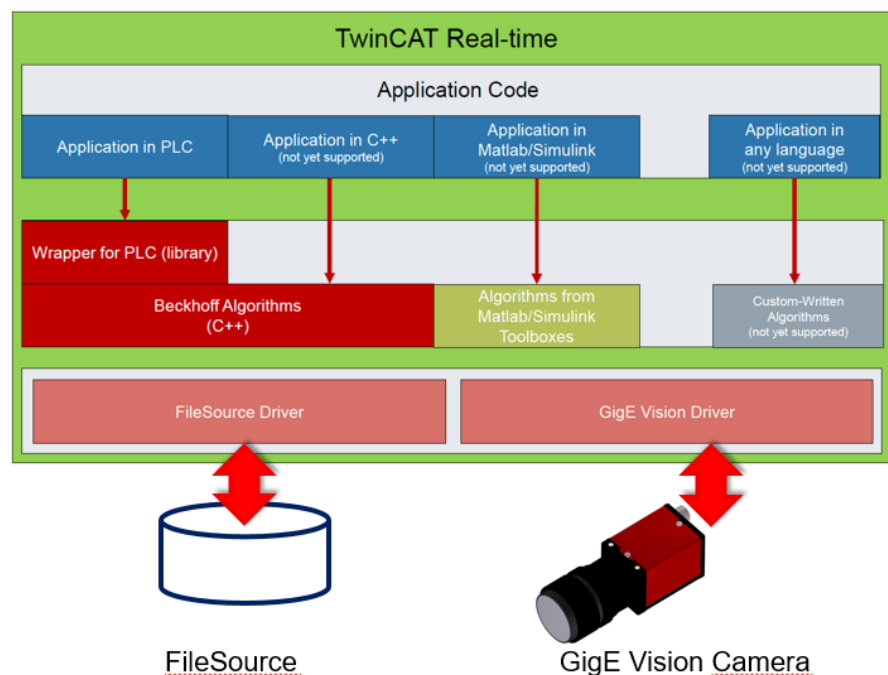
The goal of GenICam is to decouple the communication interface of the camera from the user API that is mostly a graphical UI.

2.5 Software Architecture in TwinCAT Runtime

▪ Application

▪ Algorithms

▪ Image acquisition



We could divide the software architecture in the TwinCAT Runtime into three layers: Application, Algorithms and Image acquisition.

In the Image acquisition layer, Beckhoff provides two drivers: the FileSource driver and the GigE Vision driver. The FileSource driver enables a static file to be used as the image source. The GigE Vision Driver connects to a GigE Vision camera to provide live images from the camera to TwinCAT.

All Beckhoff algorithms are written in C++ to ensure the fastest execution times possible. There is a wrapper for the PLC, so the Tc3_Vision PLC library can access to Beckhoff Algorithms. Next to the Beckhoff algorithms, it is also possible to use other algorithms such as the Matlab/Simulink Vision Toolboxes or self-written algorithms in C++. This is made possible by making the image data, provided by the Image Acquisition layer, freely accessible.

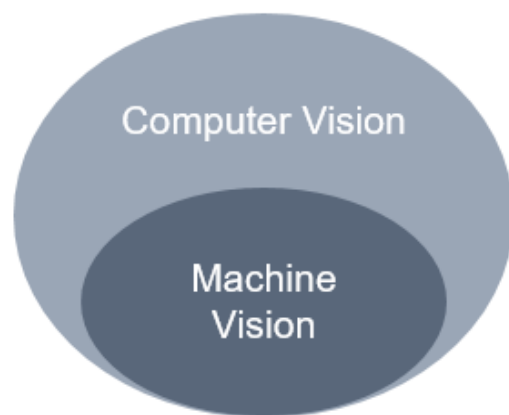
In the Application layer, the user can use and combine the before mentioned algorithms in a machine vision application. This could be a PLC application or (in the future) C++/Matlab/Simulink applications.

2.6 General functions

TwinCAT Vision focuses on the field of Machine Vision, which in turn is a specialized field of Computer Vision.

Machine Vision specializes on industrial tasks and applications such as:

- Object detection & recognition
- Check of completeness
- Position and orientation of parts
- Optical measurement
- Identification (e.g. data matrix and barcodes)
- Automated optical inspection
- Surface inspection

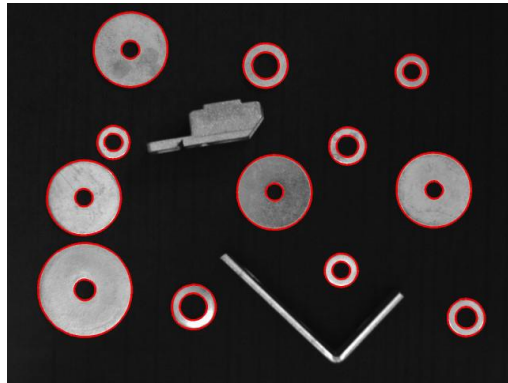


The next few topics describe typical applications that can be realized with TwinCAT Vision.

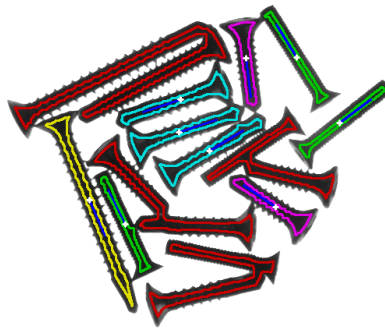
2.6.1 Object detection

Examples are detecting a part and identifying the part, such as finding circular objects.

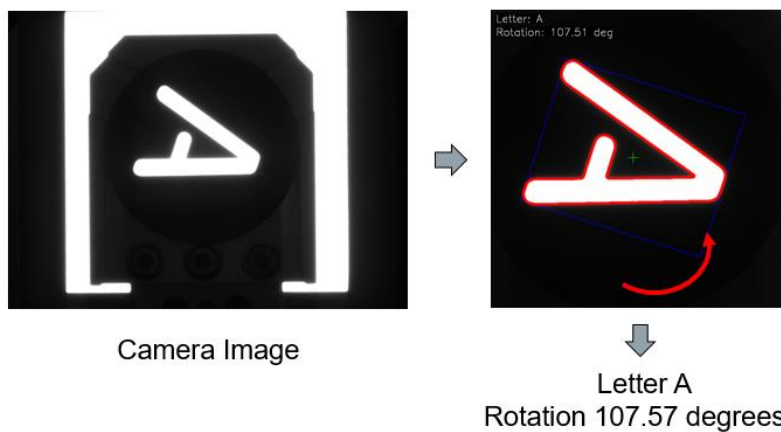
Filters can be applied based on threshold values for roundness, object size/area, colors...



Detections of screws



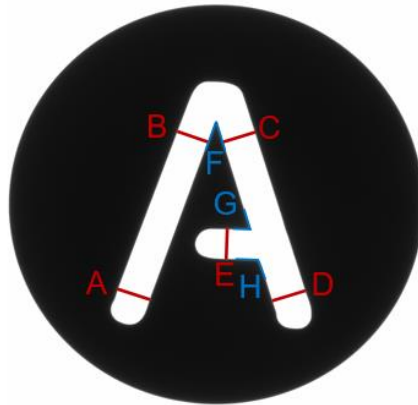
Object recognition can be done by contour matching for example. Once the object is detected, the position and orientation of the object can also be determined.



2.6.2 Measurement

Examples are:

- Length, width, height of parts
- Roundness and diameter size of drill holes
- Angle measurement
- Combined measurement for quality control



A – 3.915 mm
 B – 3.721 mm
 C – 3.932 mm
 D – 4.092 mm
 E – 3.601 mm
 F – 23.81°
 G – 63.71°
 H – 115.99°

2.6.3 Identification and code quality inspection

TwinCAT Vision can read several 1D (barcodes) codes such as EAN-8, EAN-13... as well as 2D codes such as QR codes and Data matrixes. Furthermore, label printed codes can also be evaluated in accordance with the ISO/IEC 15415 and ISO/IEC 15416 standards.

2D codes:

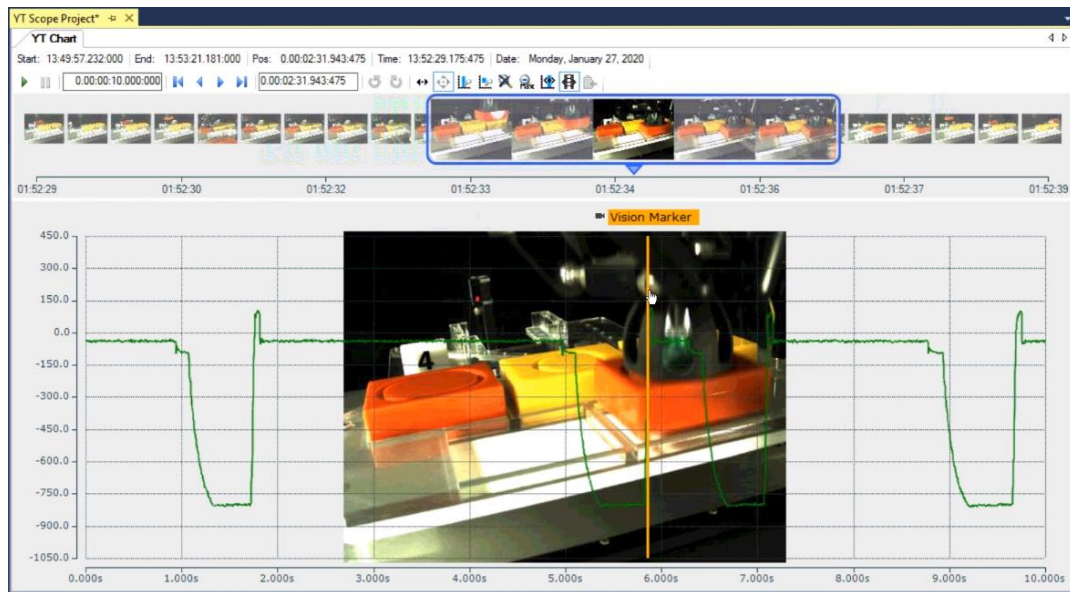


1D codes:



2.6.4 Monitoring

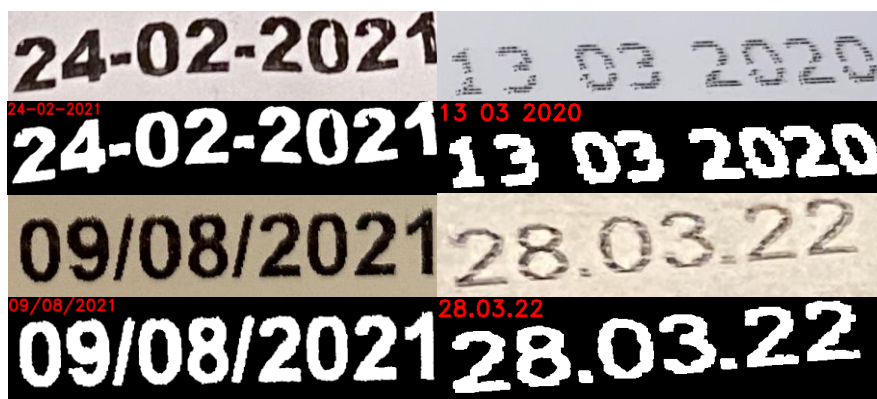
TwinCAT Vision in combination with TwinCAT HMI or TwinCAT Scope can also be used as a monitoring function to assist the machine operator during production or to help the programmer to debug certain situations by combining scoped signals with a camera stream.



2.6.5 OCR

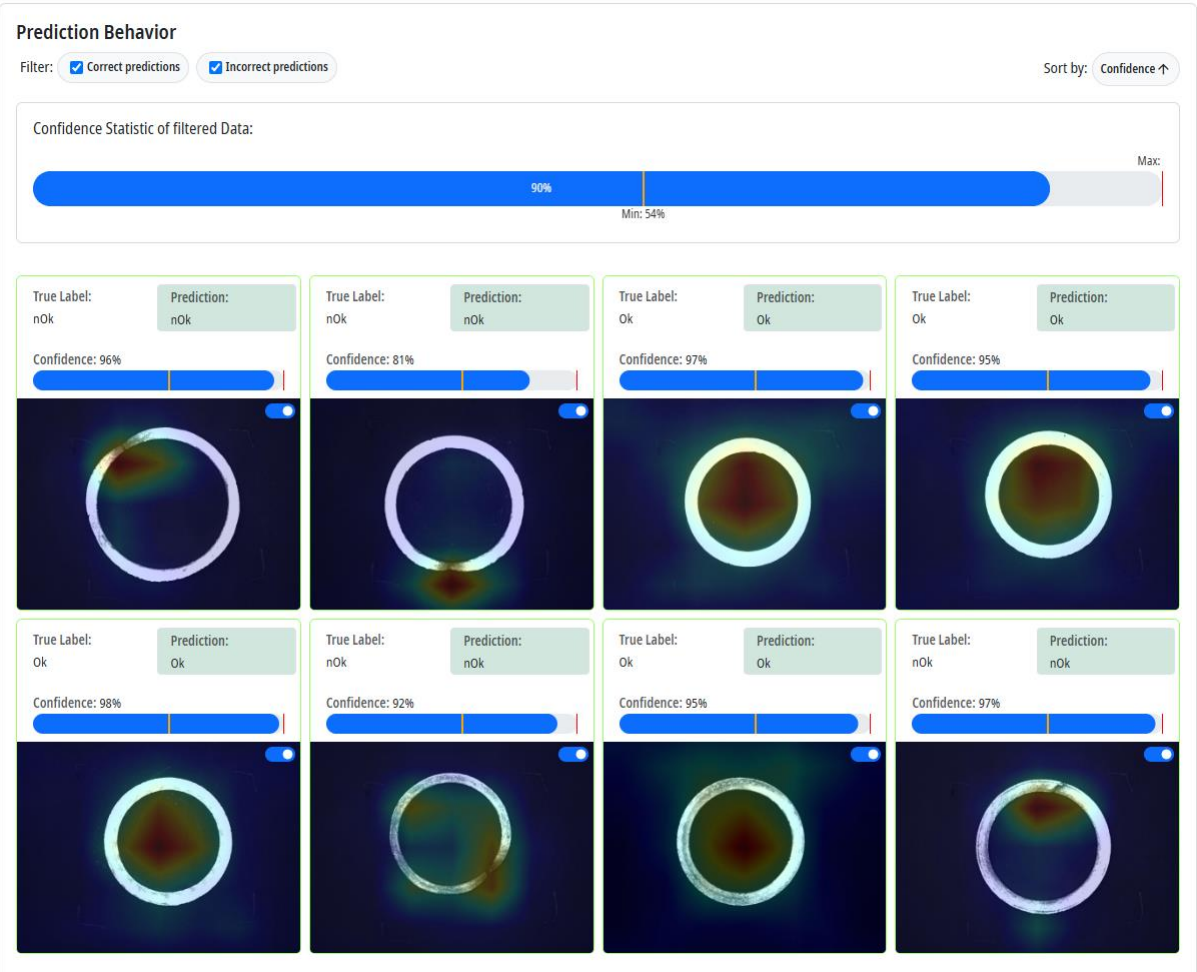
TwinCAT 3 Vision OCR contains functions for optical character recognition (OCR). The functions identify characters in an image and return the recognized character string.

The functions are based on pre-trained machine learning models. No additional settings or training are required. Application examples include verification of best-before dates and batch numbers.



2.6.6 Anomaly detection

With the TwinCAT Machine Learning Creator now you can also train your own models to be able to detect anomalies in a product.



3. Product overview

The TwinCAT Vision Product series is an expanding line-up of both software and hardware products.

3.1 Software

The TwinCAT Vision software products are categorized under the TF7xxx name and are separated into two main groups: Connector licenses to connect cameras and Algorithm licenses.

3.1.1 Overview

Hardware licenses

To connect GigE Vision cameras to a TwinCAT 3 system, you need the TF700x TC3 GigE Vision Connector license(s). This product is GigE Vision certified.

By combining different TF700x licenses, up to 16 cameras can be connected to one runtime. As always the case with TwinCAT 3, 7-day test licenses are available.

TwinCAT 3 GigE Vision		
TF7000	TC 3 GigE Vision Connector Base	Always required. Basics for the connection, configuration and image acquisition with GigE Vision cameras. It contains 2 camera connections.
TF7001	TC 3 GigE Vision Connector Pack 2	TwinCAT 3 GigE Vision Connector Pack 2
TF7002	TC 3 GigE Vision Connector Pack 4	TwinCAT 3 GigE Vision Connector Pack 4
TF7003	TC 3 GigE Vision Connector Pack 8	TwinCAT 3 GigE Vision Connector Pack 8

The TwinCAT 3 Vision Beckhoff Camera Connector offers the option of integrating Beckhoff cameras directly into the TwinCAT architecture.

TwinCAT 3 Vision Beckhoff Camera		
TF7020	TC 3 Vision Beckhoff Camera Connector	Establishes the basis for communication and provides up to 64 camera connection points.

The TwinCAT 3 Vision Algorithms are divided into separate categories based on functionality and complexity.

TwinCAT 3 Vision Algorithms		
TF7100	TwinCAT 3 Vision Base	Library comprising a variety of functions covering the basics of image processing and camera integration in the PLC.
TF7200	TwinCAT 3 Vision Matching 2D	Functions for detection and matching objects based on learned templates, contours, key points and other references.
TF7250	TwinCAT 3 Vision Code Reading	Functions for reading multiple types of 1d and 2d codes.
TF7255	TwinCAT 3 Vision Code Quality	Functions for quality evaluation of various 1D and 2D codes. Evaluation is carried out in accordance with the ISO/IEC 15415 and ISO/IEC 15416 standards, for label printed codes .
TF7260	TwinCAT 3 Vision OCR	Functions for optical character recognition (OCR). The functions identify characters in an image and return the recognized character string. Application examples include verification of best-before dates and batch numbers .
TF7300	TwinCAT 3 Vision Metrology 2D	Functions for sub-pixel accurate detection of edges, ellipses, arcs as well as lengths, distances, diameters, angles and coordinates. Functions for camera calibration and transformations between image coordinates and world coordinates.
TF7800	TwinCAT 3 Vision Machine Learning	Provides an integrated machine learning (ML) solution for vision-specific use cases. Both the training and the implementation of the machine learning models take place in real time, and they even help machines to learn sophisticated data analyses automatically. Application examples include object recognition/detection, sorting, quality control, and process monitoring.
TF7810	TwinCAT 3 Vision Neural Network	High-performance execution module (inference engine) for trained neural networks. The neuronal networks are trained in established frameworks such as PyTorch, TensorFlow, or MATLAB®. In contrast to classic ML models, image data is transferred directly, and feature extraction takes place in the models.

3.1.2 Licensing

The TwinCAT 3 functions can be activated as a full version or as a 7-day test version.

The licensing **starts at platform level 50** and goes all the way up to platform level 90+.

TC1200 TwinCAT PLC is required for the TF7100 TC3 Vision Base PLC library. All the other TF7xx0 library extensions require the TF7100 license.

Vision		Required
TF7000	TwinCAT 3 GigE Vision Connector Base	TC1200/TC1300
TF7001	TwinCAT 3 GigE Vision Connector Pack 2	TF7000
TF7002	TwinCAT 3 GigE Vision Connector Pack 4	TF7000
TF7003	TwinCAT 3 GigE Vision Connector Pack 8	TF7000
TF7020	TwinCAT 3 Vision Beckhoff Camera Connector	TC1200/TC1300
TF7100	TwinCAT 3 Vision Base	TC1200
TF7200	TwinCAT 3 Vision Matching 2D	TF7100
TF7250	TwinCAT 3 Vision Code Reading	TF7100
TF7255	TwinCAT 3 Vision Code Quality	TF7100
TF7260	TwinCAT 3 Vision OCR	TF7100
TF7300	TwinCAT 3 Vision Metrology 2D	TF7100
TF7800	TwinCAT 3 Vision Machine Learning	TF7100
TF7810	TwinCAT 3 Vision Neural Network	TF7100

4. TwinCAT Vision in the XAE

TwinCAT Vision is fully integrated in the TwinCAT 3 development environment. Essentially, TwinCAT Vision extends the TwinCAT system by a further node, below which the Vision components can be created and configured. The following configuration elements are used:

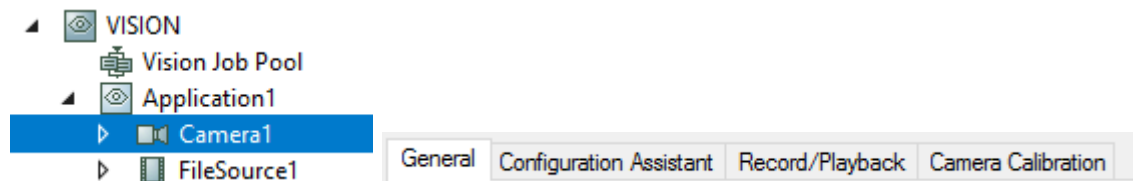


- The **VISION node** provides general configuration options that affect all Vision components on the system. It includes all other configuration elements.
- The **Vision Job Pool** can be used to define job tasks for parallel execution of Vision functions. Each system has a single Vision Job Pool.
- An **application node** encapsulates several Vision devices (GigE Vision cameras and File Sources) to form an applicative unit. The purpose of application nodes is consolidation of thematically related Vision devices. The entire configuration of an application can be exported and imported. Each TwinCAT project can have several applications.
- A **GigE Vision camera node** contains the configuration of a physical GigE Vision camera. Each application can have several GigE Vision camera nodes.
- A **File Source node** contains the configuration for an interface between images on the hard disk of the target system and an image acquisition module within the PLC. Each application can have several File Source nodes.

4.1 Camera

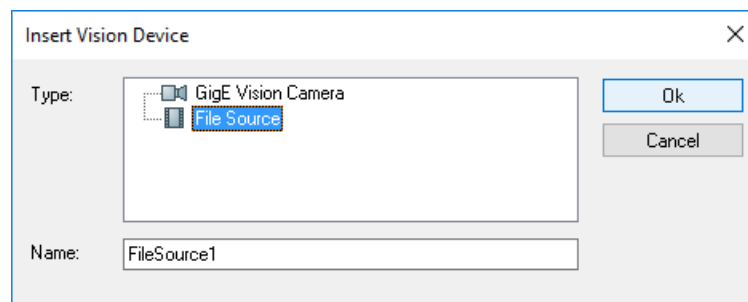
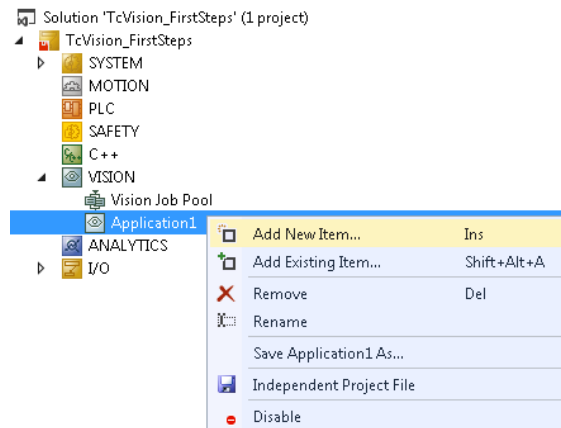
The configuration of a GigE Vision camera is done in its corresponding camera node. The camera node consists of four tabs: General, Configuration Assistant, Record/Playback and Camera Calibration.

The configuration, calibration etc. are not discussed in this document but are part of the training course. In this quickstart we will concentrate on the file source.

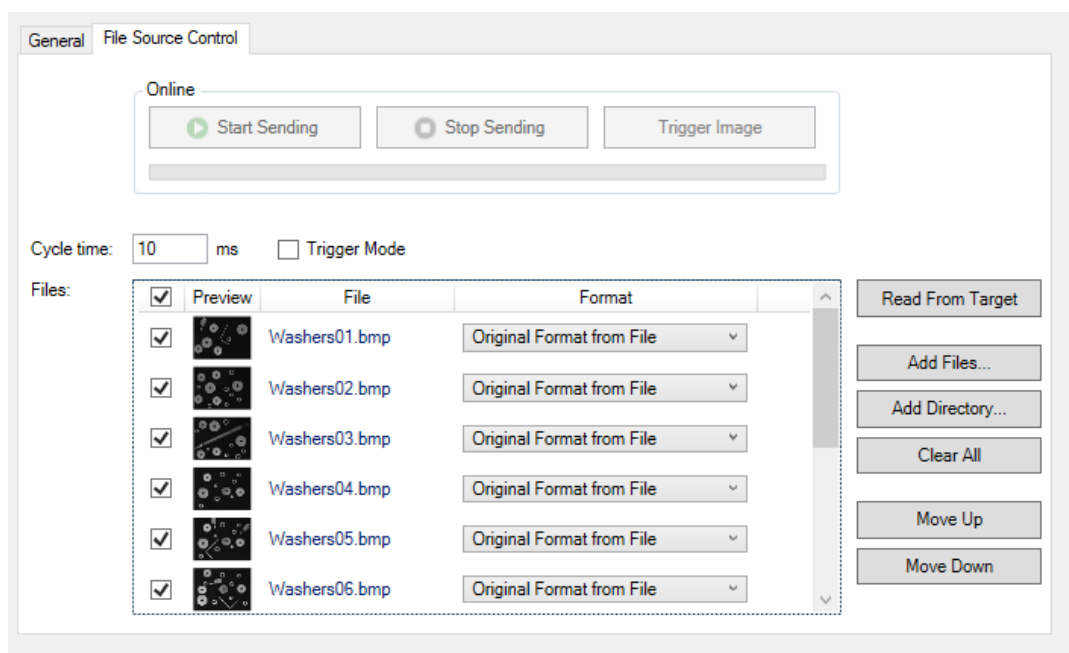


4.2 File Source Control

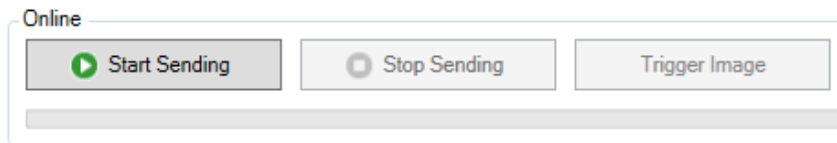
A File Source can be added to the Vision application node by right clicking, add new item and selecting the File Source option.



The configuration page of the File Source Control looks like:



Here you can add images to the File Source Control, Start/stop the image streaming or trigger each image individually.



Alternatively, the commands can also be sent from within the PLC with the **FB_VN_FileSourceControl** function block. The necessary methods are **StartAcquisition()**, **StopAcquisition()** and **TriggerImage()** respectively.

The File Source Control has two modes for providing images:

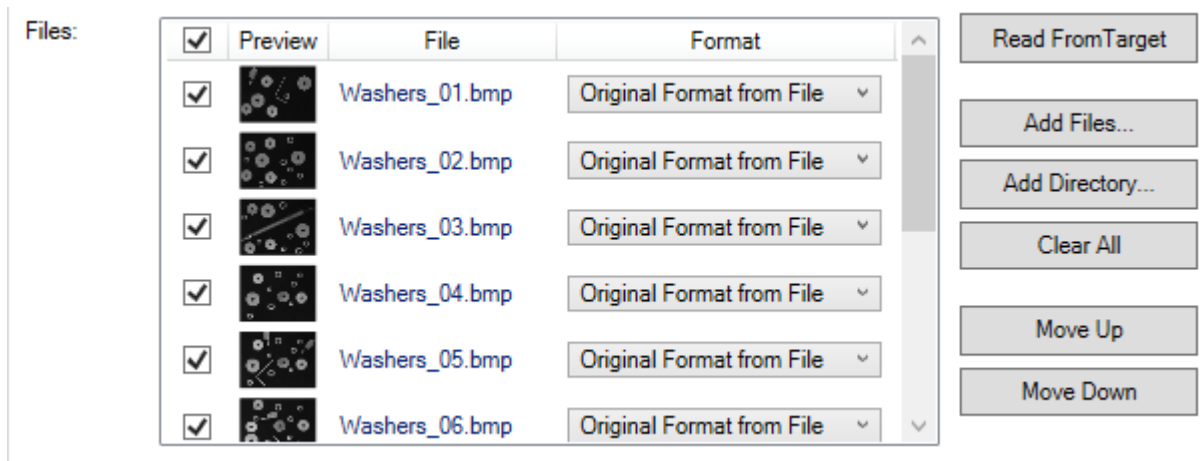
1. **Streaming mode:** Like a streaming camera, new images are provided in a specified cycle.
2. **Trigger mode:** A new image is provided with a trigger signal. This can be triggered either by a software trigger or by manually actuating a button.

The mode selection is done by these options: Cycle time: ms ☐ Trigger Mode

Cycle Time: if no Trigger mode is selected, the images are sent to the real-time context at the Cycle Time interval. The order of the images according to the file list is taken into account. This parameter is comparable to the frame rate of a camera.

Trigger Mode: if Trigger Mode is selected, an image is sent after a corresponding software trigger is called. With the standard software trigger, the order of the images according to the file list is observed. Alternatively, a specific image can be triggered based on the file name.

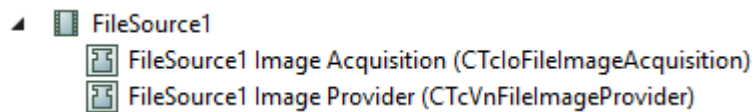
The images to be used by the File Source Control are managed via a file list and the associated controls:



An important setting to consider here is the **format** of an image. By default, images are sent in the saved format. Alternatively, it can be specified to convert this into a 1-channel grayscale image or 3-channel RGB image in advance and then send it. **We recommend that you explicitly specify the format for each image.**

Images from the development system can be selected via a file explorer. These are then copied to the target system in the background, so that they are available there independently of the development system. Please note that only files with unique names can be used. In addition, only image formats, i.e. not video formats, can be added here. Unless configured otherwise, the image files are stored on the target system in the path **"C:\Users\Public\TcVision\FileSources"**.

The base path can be changed in the Image Acquisition TcCOM object of the File Source:



The **TcIoFileImageAcquisition** object coordinates the loading of images from the hard drive for FileSource instances.

The Image Provider TcCOM object (**TcVnFileImageProvider**) handles the interfacing to the PLC.

5. Prerequisites for the exercises

5.1 Cloud environment

The use of the dedicated cloud environment is for INTERNAL USE ONLY.

For login and credentials ask the vision team.

5.2 Local engineering station

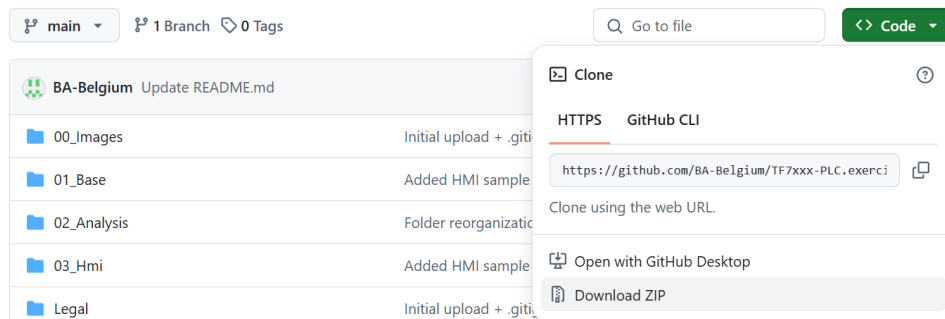
Installed software

- [TwinCAT](#): Version $\geq 3.1.4026$
- [TwinCAT HMI](#): Version $\geq 1.14.3$
- [TF7xxx](#): version $\geq 4.0.7.4$

The template sources, complete projects and images for this quickstart can be found on the Beckhoff Belgium github account.

<https://github.com/BA-Belgium/TF7xxx-PLC.exercises.quickstart.plc>

For the exercises: open the link, download the source code (download zip) and extract the zip in the desired location on your development station.



6. Project sample - Basic Vision Project

6.1 Objective

In this section you will discover how to create a basic TwinCAT Vision Project. This project will be used as a template for the following sections.

This template will help you to understand how to develop a vision application inside a TwinCAT 3 PLC Project and explains some best practices.

6.2 Creating a TwinCAT XAE solution for a vision application

TwinCAT Vision is fully integrated in the TwinCAT system. A standard TwinCAT project is therefore the starting point for every TwinCAT Vision application:

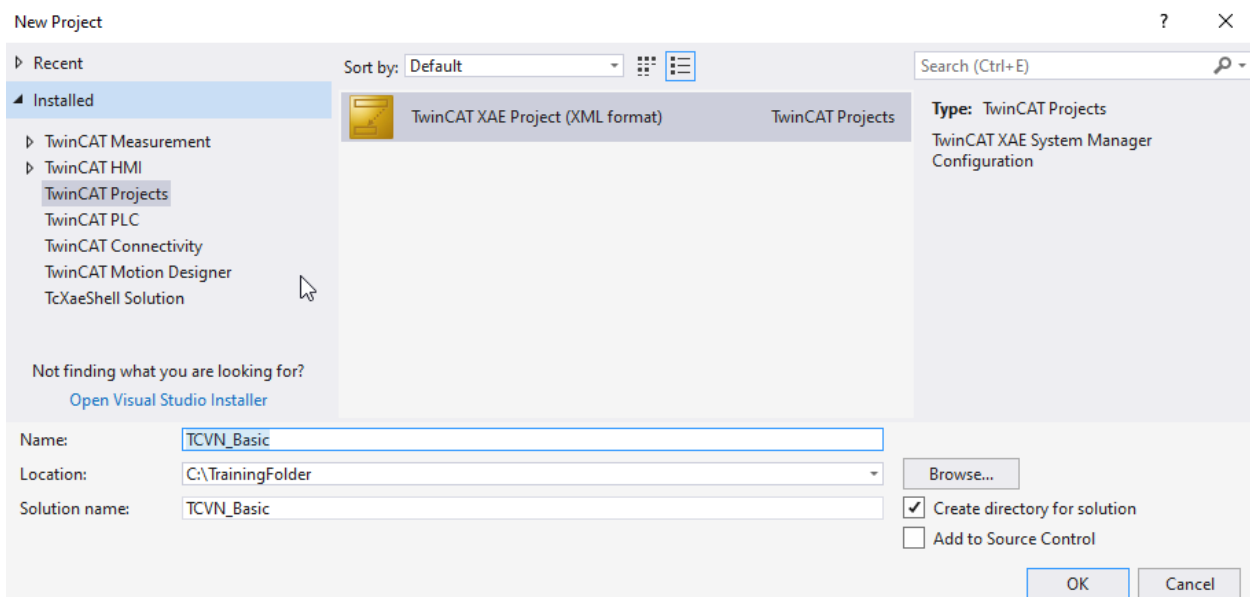
1. Start TwinCAT XAE

TwinCAT Vision can be created with the VS Shell or a full version of VS

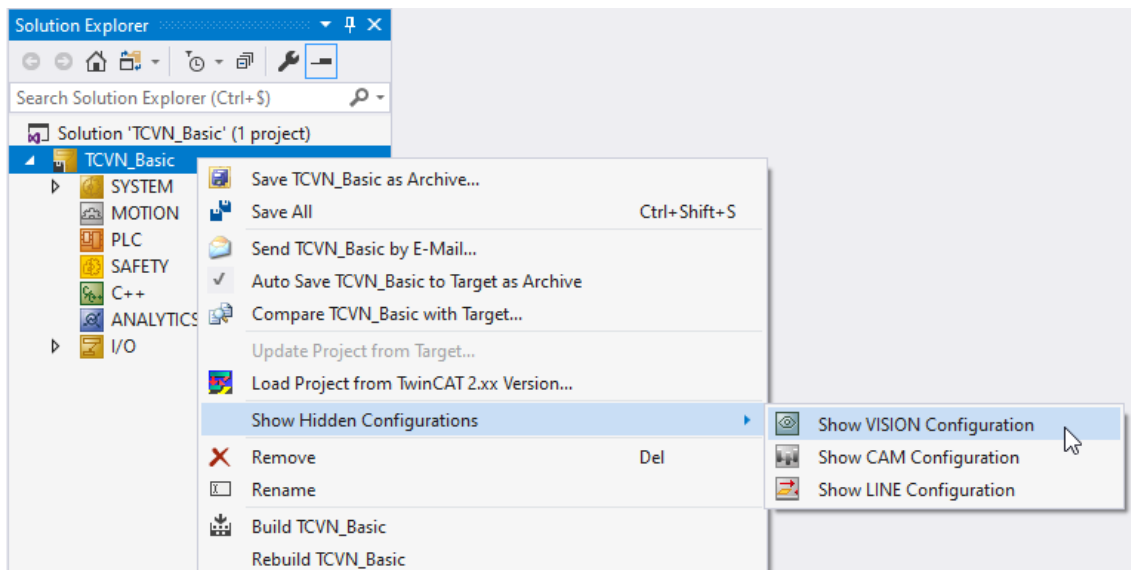
2. Create a TwinCAT XAE project

Name : TCVN_Basic

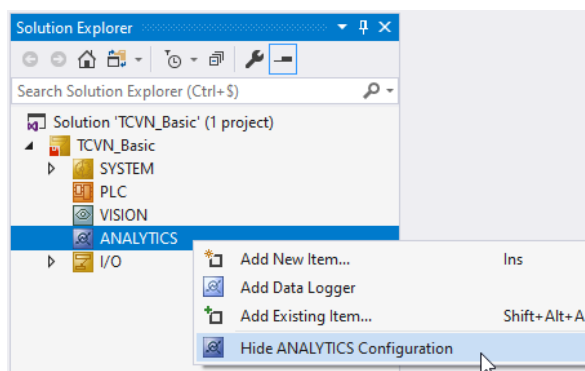
Location : C:\TrainingFolder



3. Enable the vision system configuration in the Solution Explorer



4. Optionally hide the modules that aren't needed.



5. Connect to the Runtime system (in the case the runtime is the Local runtime, then avoid this step).

6. Configure the System Real-Time :

Increase router memory : The router memory should be sized according to the memory capacity of the system (max 1024MB). The router memory is dynamically used by the vision software as storage.

Selection of a CPU core and a task : In multi-core systems, TwinCAT enables a distinction between Windows cores and isolated cores. With Windows cores, the operating system and the TwinCAT application share the processor time. The real-time portion can be limited to between 10% and 90%. In contrast, isolated cores are fully available to the TwinCAT application. For this reason, the use of isolated cores is recommended for Vision applications. Divided cores can also be used without problems for the initial testing.

Create a new Task „VisionTask“ at 40ms.

If available, use an isolated core for the Vision Task (VisionTask) and the vision acquisition task (VnTask ImAcq).

REMARK: VnTask ImAcq will only appear once you have added an image source (file or camera). See the paragraph vision node – file source configuration of this exercise.

Core	RT-Core	Base Time	Core Limit	Latency Warning	Core Memory	Core Frequency
0 (Sha...	☐					Base Frequency
1 (Sha...	☐					Base Frequency
2 (Sha...	☒ Default	1 ms	80 %	(none)	512 KB with 2 KB limit	Base Frequency
3 (Isol...	☒	1 ms	100 %	(none)	512 KB with 2 KB limit	Base Frequency

Object	RT-Core	Base Time (ms)	Cycle Time (ms)	Cycle Ticks	Priority
VisionTask	Core 3	1 ms	10 ms	10	1
VnTask ImAcq (FileSource1)	Core 3	1 ms	10 ms	10	5
I/O Idle Task	Default (2)	1 ms	1 ms	1	11
PlcTask	Default (2)	1 ms	10 ms	10	20
Aux Task	Default (2)	1 ms	(none)	0	50

REMARK: the realtime configuration should be set appropriate to the target system and can be different than proposed above.

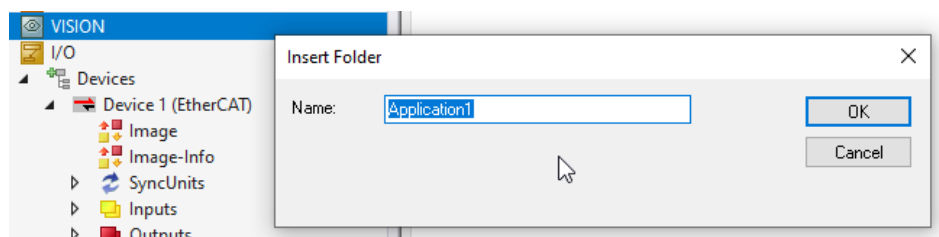
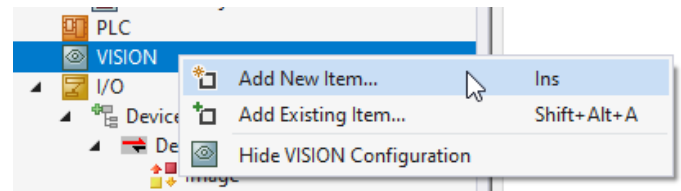
6.3 Vision node – File source configuration

For the exercise we will use a file source that can be used for most of the functions of the Tc3 Vision library. Only those functions that use camera calibration parameters cannot be used for this type of vision device.

1. Create a Vision folder

The folder contains other devices such as Camera or FileSource.

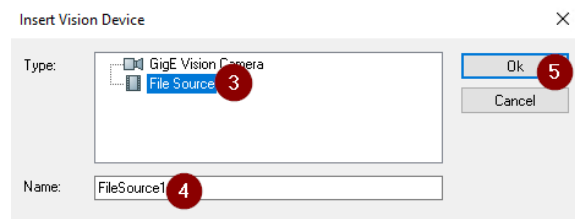
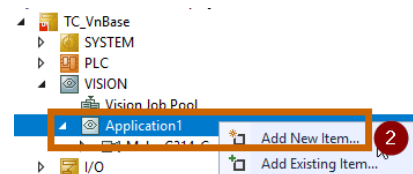
Additional folders for structuring the project can be created.



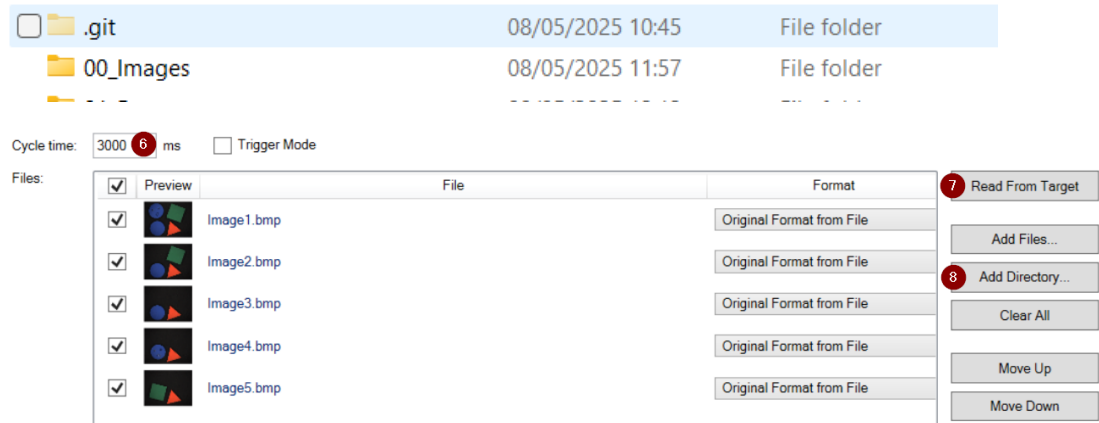
2. Insert a new Vision Device of type File Source

To configure a File Source Control the following must be done :

1. The PLC must be in Config Mode
2. Go to the Application under the Vision node and do right click **Add New Item...**
3. Select **File Source** as Vision Device.
4. Choose suitable name.
5. The click OK.

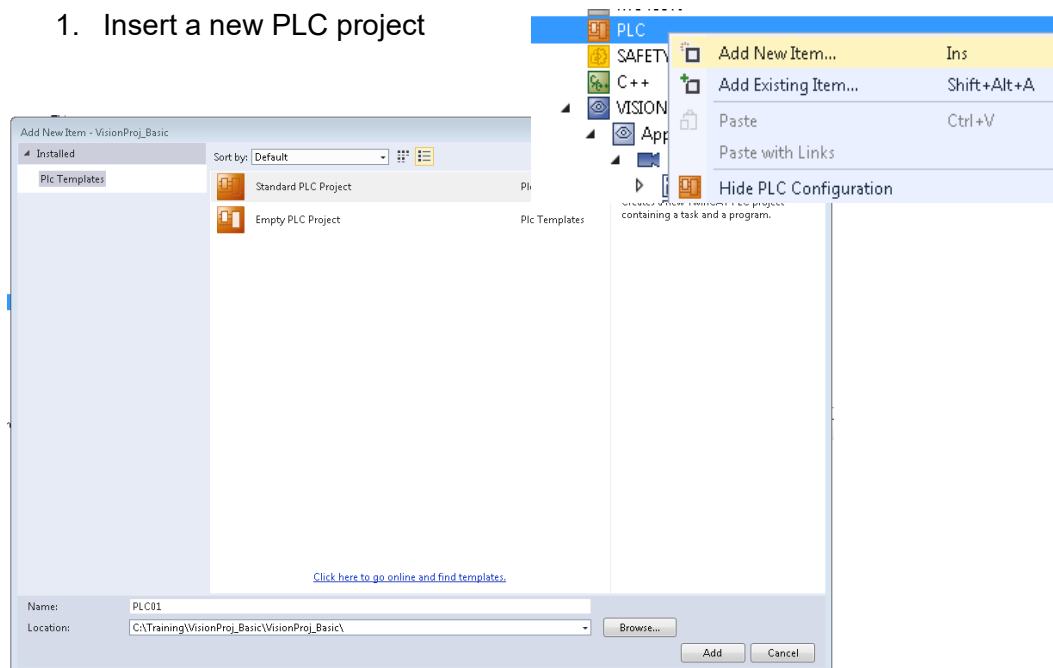


6. Configure **File Source Control** double clicking on it in the project solution. Choose a suitable cycle time (3000ms for the exercise).
7. If 'add' buttons are greyed out, click on 'Read From Target'
8. Load the images (in the repository 00_Image folder) necessary for the exercise

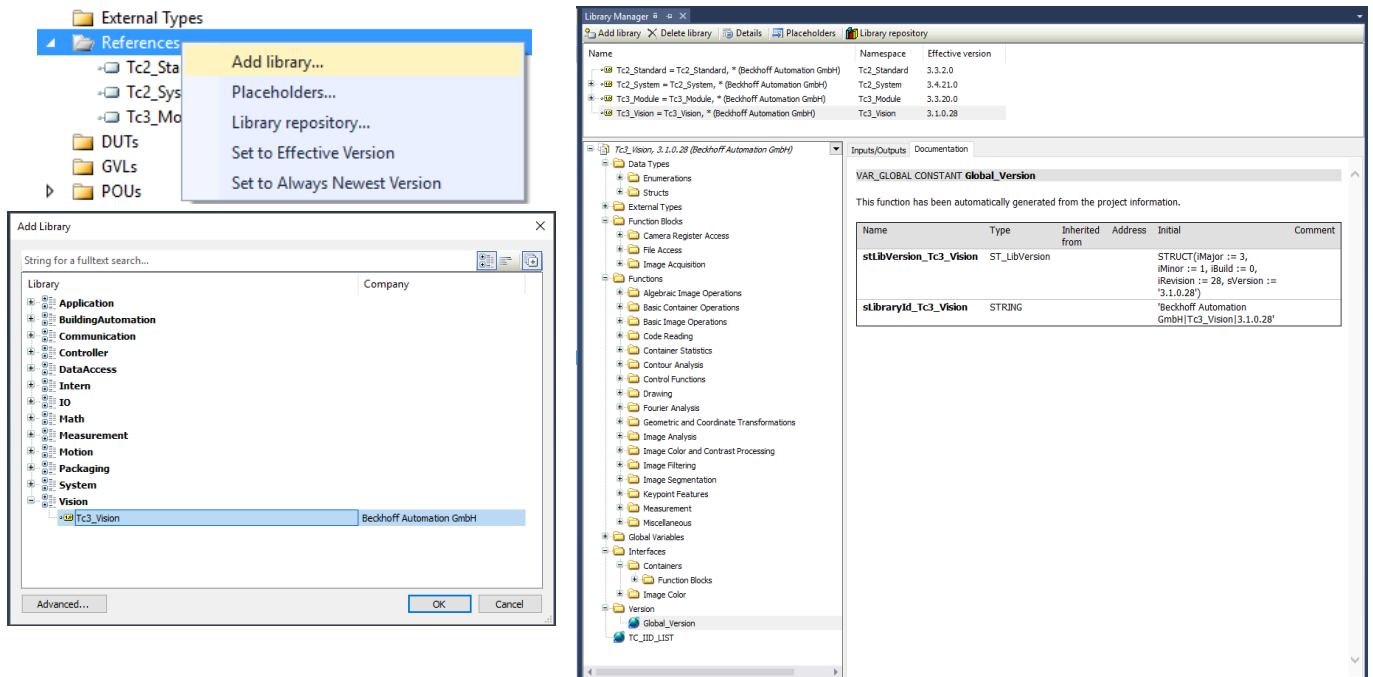


6.4 Base PLC project for vision applications

1. Insert a new PLC project



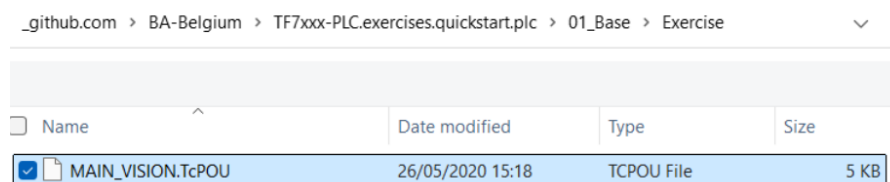
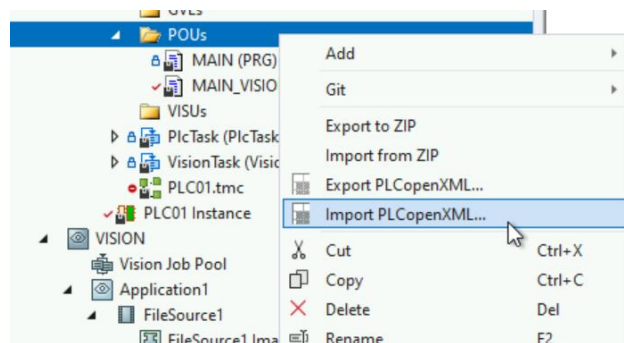
2. Insert the vision library Tc3_Vision



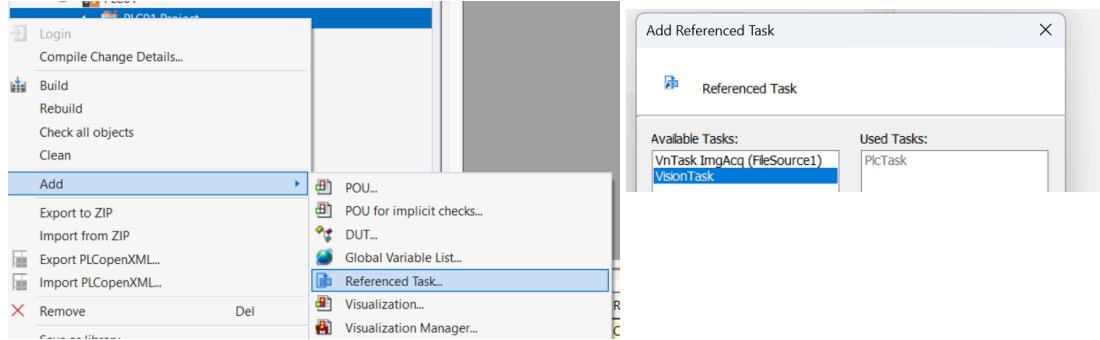
3. For simplicity, we will use templates that need to be imported into the newly created plc program.

The template for the main vision loop that handles the image acquisition can be found and imported from the repository folder 01-Base\Templates

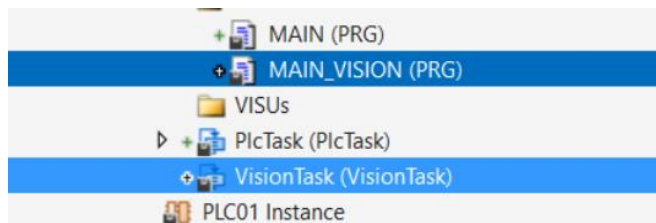
Right click on the plc project folder POU and choose Import PLCopenXML -> select MAIN_VISION.XML



4. Add a referenced task to the plc, select the vision task



5. Drag and drop the imported MAIN_VISION (PRG) to the created referenced task



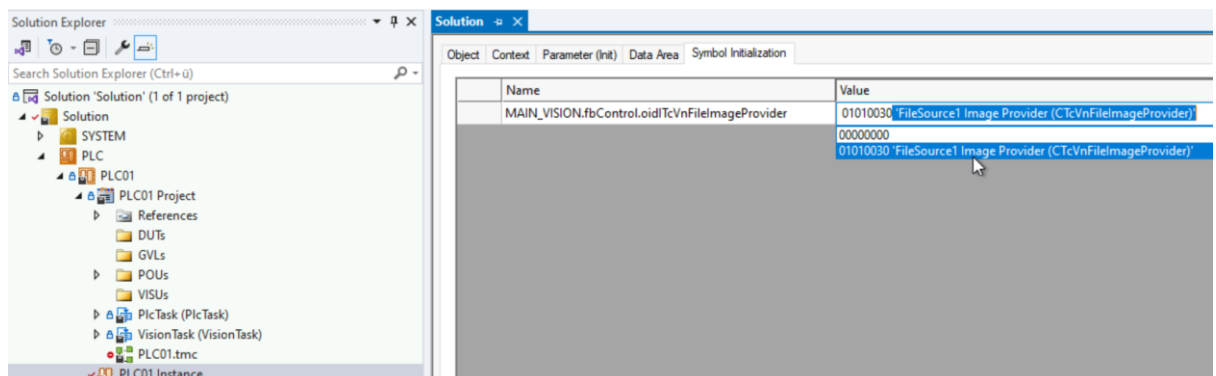
6. For this exercise to work, it must be linked to an instance of **FB_VN_FileSourceControl** in the PLC project.

```

PROGRAM MAIN_VISION
VAR
    eControlState      : ETcVnCameraState;
    fbControl           : FB_VN_FileSourceControl;    // File Source Control
    // fbControl        : FB_VN_GevCameraControl;    // GigE Vision Camera Control
    // fbControl        : FB_VN_SimpleCameraControl;  // File Source OR Camera Control

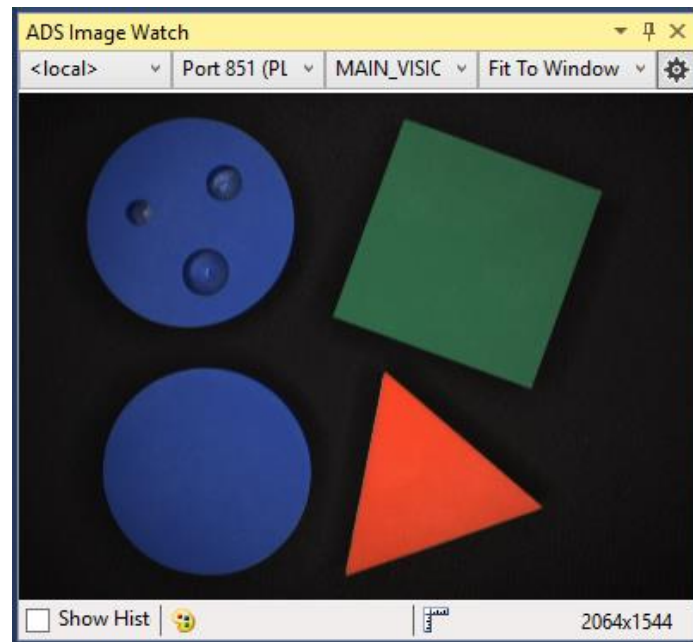
```

7. Build the solution, reference the FB_VN_FileSourceControl FB instance in the plc to the 'File Source Control' image provider interface



8. Activate the configuration, this will start TwinCAT + send the pictures to the XAR

9. Login to the plc and press run if necessary (if not set to autostart)
Running the plc code will trigger image acquisition and fetch the image into containers that can be used in the vision plc program.
10. Open the **ADS Image Watch** window (View > Other Windows > ADS Image Watch) and select the correct target, choose port 851 and select **MAIN_Vision.iplImageDisp** to view the input images provided by FileSource1.
The image should change every 3 seconds like we configured in the File Source Control.



6.5 Conclusion

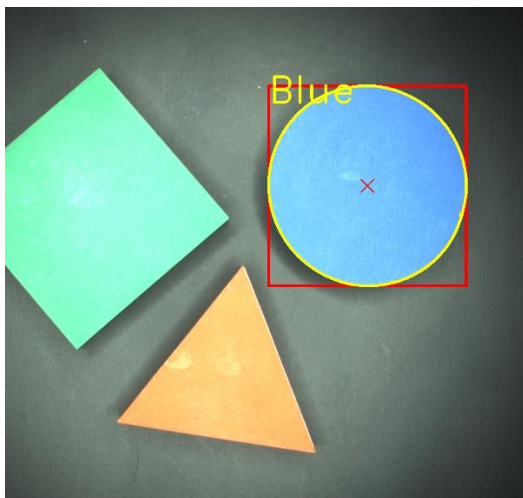
What we implemented in this exercise is a basic implementation in the plc that fetches incoming messages from the configured source (file source).
in this exercise we did not manipulate or process the incoming pictures, we just display them with the image watch.

7. Project sample - Image analysis: Colors and contours

7.1 Objective

This project sample will cover:

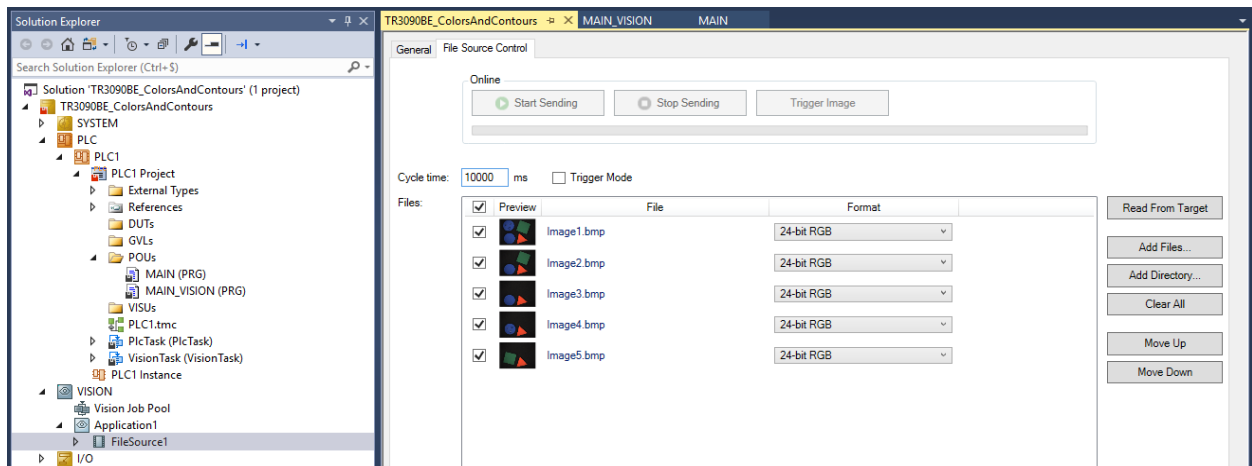
- Colored images and color spaces
- PLC variables for representing colors
- Simple color segmentation and contour recognition
- Iterators
- Configuration of a Result Image
- Draw and text functions



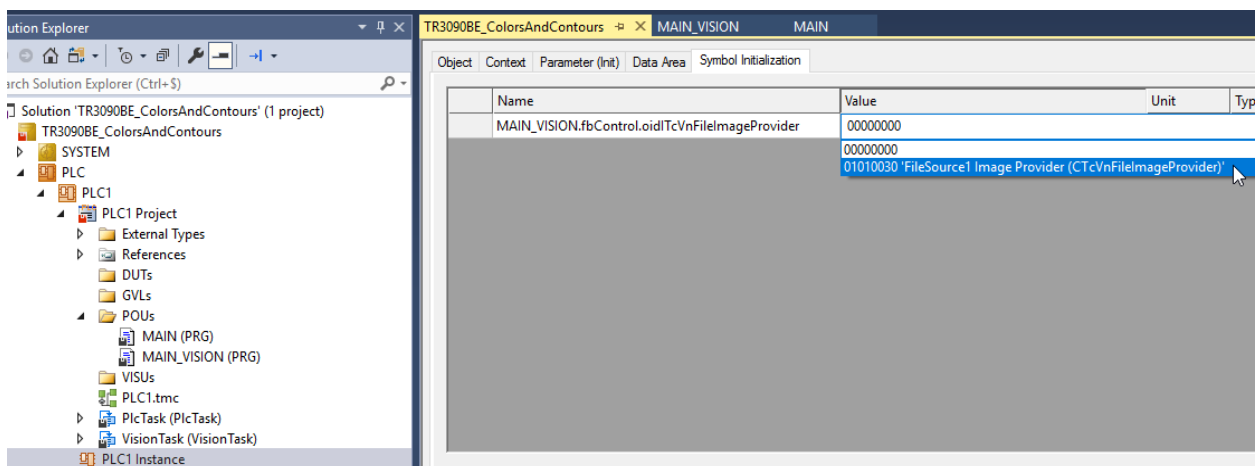
7.2 Initial setup (see base exercise for details)

The starting point for this sample is **Base.tszip**

1. Open the tszip-archive located in the 02_Analysis\Templates folder of the repository. Optionally you can give your solution a fitting name and directory.
2. Make sure the target XAR is in config mode.
3. In the configured File Source Control, set the **Cycle Time** to 10000ms and add Image1.bmp until Image5.bmp via **Read From Target > Add Files...**
The images are in the 00_Images folder of this repository.
4. Rebuild the solution: via clean solution, rebuild solution



5. Link the FileSource1 Image Provider to the fbControl instance of the PLC project. You can do this in the **PLC1 Instance > Symbol Initialization**. If the fbControl instance is not available on the list, clean & (re)build the PLC project first.



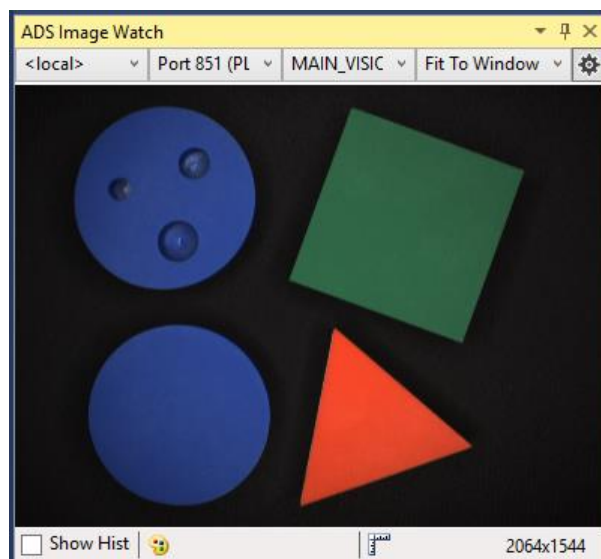
6. Go to the Real-Time settings of the TwinCAT project and check the settings:
 - Make sure the Configured Size of the **Router Memory** is sufficiently large enough (go for minimum 256MB)
 - Read out the number of cores for your target and **configure the cores and tasks accordingly**. Ideally, a minimum of two cores are to be available for TwinCAT. Ideally, one core is exclusively used for the VisionTask and the ImgAcq task.

Example configuration

Core	RT-Core	Base Time	Core Limit	Latency Warning	Core Memory	Core Frequency
0 (Sha...)	☐					Base Frequency
1 (Sha...)	☐					Base Frequency
2 (Sha...)	☒ Default	1 ms	80 %	(none)	512 KB with 2 KB limit	Base Frequency
3 (Isol...)	☒	1 ms	100 %	(none)	512 KB with 2 KB limit	Base Frequency

Object	RT-Core	Base Time (ms)	Cycle Time (ms)	Cycle Ticks	Priority
VisionTask	Core 3	1 ms	10 ms	10	1
VnTask ImgAcq (FileSource1)	Core 3	1 ms	10 ms	10	5
I/O Idle Task	Default (2)	1 ms	1 ms	1	11
PicTask	Default (2)	1 ms	10 ms	10	20
AuxTask	Default (2)	1 ms	(none)	0	50

7. Activate the TwinCAT project and run/login to the PLC project. If you are experiencing a runtime crash immediately, then probably the router memory size is not set high enough.
8. Open the **ADS Image Watch** window (View > Other Windows > ADS Image Watch) and select the correct target, choose port 851 and select **MAIN_Vision.iplImageDisp** to view the input images provided by FileSource1. The image should change every 10 seconds as we configured in the File Source Control.



7.3 Implementation

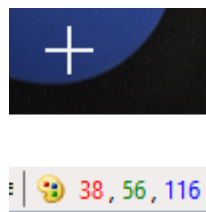
In scope of this exercise, we will mainly work with three images in the PLC projects:

- **ipImage**: the original image provided by the camera/fileSource
- **ipImageWork**: working copy where we deploy the algorithms on
- **ipImageResult**: result color copy where we draw results on to show in an HMI for example

1. We want to filter out only the blue parts of the image. Therefore, we could use the **F_VN_CheckColorRange** function. The result of this function is a one-channel image where the pixels are set to 255 (white) if the pixel value of the input image lies in the given color range and to 0 (black) if it does not.

Color-variables in TwinCAT Vision have the type **TcVnVector4_LREAL**, which is basically an array of four LREAL values representing the RGBA values of a color. It is sufficient to provide only the RGB values (and leave the A value to 0).

To have an idea what RGB values we could use for the blue parts, check the ADS Image Watch:



2. Open MAIN_VISION (PRG)
3. Add two variables in the declaration window (*aLowerBounds* and *aUpperBounds*) of TcVnVector4_LREAL and determine a suitable range to filter the blue parts.

```
// working variables
aLowerBounds      : TcVnVector4_LREAL := [25,25,80,0]; // R,G,B,A
aUpperBounds      : TcVnVector4_LREAL := [100,100,160]; // (transparency A not used here)
```

4. Implement the F_VN_CheckColorRange function in the code window:

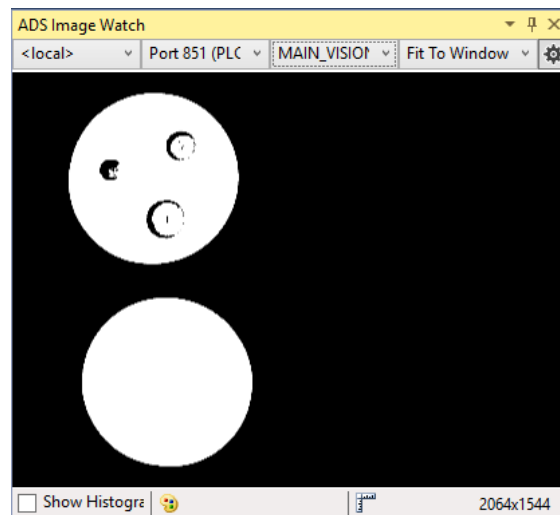
```
// *****
// Add Vision Algorithms
// *****

hr := F_VN_CheckColorRange (
    ipSrcImage:= ipImageWork,
    ipDestImage:= ipImageWork,
    aLowerBounds:= aLowerBounds,
    aUpperBounds:= aUpperBounds,
    hrPrev:= hr);
```

5. Cut and paste the `F_VN_CopyIntoDisplayableImage` debug code (see current base code) behind the `CheckColorRange` algorithm. We can cut and paste this code line wherever we need to see intermediate algorithm results for debug purposes.

```
(* debug ==> Copy work Image to a displayable Work-Image
   ipImageWork is still available after copy *)
hr := F_VN_CopyIntoDisplayableImage(ipSrcImage:= ipImageWork, ipDestImage:= ipImageWorkDisp, hrPrev:= hr);
```

6. Download the PLC code and check the work image in the ADS Image Watch. Adjust your upper and lower bound values until you get a result similar like this:



7. In this filtered work image, we can now easily find the white objects by using the function `F_VN_FindContours`. This function will return a list of all found contours. As the size of this list is not known beforehand, we need a dynamic container variable type: `ITcVnContainer`. Create a variable `ipContourList`:

```
ipContourList : ITcVnContainer;
```

Now implement `F_VN_FindContours`:

```
// Find contours
hr := F_VN_FindContours(ipSrcImage:= ipImageWork, ipContours:= ipContourList, hrPrev:= hr);
```

8. The contour list will now contain all contours found in the image. To access each individual contour, we must iterate through the list. This can be achieved with an `ITcVnForwardIterator`. To create a forward iterator for the contour list, use the function `F_VN_GetForwardIterator`. Create an iterator `ipIterator`:

```
ipIterator : ITcVnForwardIterator;
```

Implement `F_VN_GetForwardIterator`:

```
// create the iterator to loop through the elements
hr := F_VN_GetForwardIterator(ipContainer:=ipContourList, ipIterator:=ipIterator, hrPrev:=hr);
```

9. With this iterator, we can access each contour in the list individually. To implement this, use a while loop that checks if the iterator is at the end of the list. Inside the

while loop we have to increment the iterator (otherwise you will have an infinite loop!) and we extract each contour from the list.
Create a variable *ipSingleContour* to store your single contour:

```
ipSingleContour : ITcVnContainer;
```

Implement the while loop:

```
// iterate over the contour list
WHILE SUCCEEDED(hr) AND_THEN ipIterator.CheckIfEnd() <> S_OK DO
  // extract a single contour out of the list
  hr := F_VN_GetContainer(ipIterator:= ipIterator, ipContainer:= ipSingleContour, hrPrev:= hr);
  // increment the iterator
  hr := F_VN_IncrementIterator(ipIterator:= ipIterator, hrPrev:= hr);
END_WHILE
```

Notice the **AND_THEN** keyword, which will only check the statement after the **AND_THEN** when the statement before is true! This way we make sure that *ipIterator* is not null when we call its method *CheckIfEnd()*, which would otherwise cause a crash.

10. If we would the number of elements in the contour list, we would most probably notice that there are much more contours than we would expect. This is because the FindContours algorithm also finds the very small contours (even if they are only some pixels big) if the image is not clean enough. One way to take care of this is to filter the contours based on their area size. To determine the area of a contour use **F_VN_ContourArea**. Create a LREAL variable, *fArea*, to store the area of the contour:

```
fArea : LREAL;
```

Filter the contour on its area size adjacent to the code in the while loop

```
WHILE SUCCEEDED(hr) AND_THEN ipIterator.CheckIfEnd() <> S_OK DO
  // extract a single contour out of the list
  hr := F_VN_GetContainer(ipIterator:= ipIterator, ipContainer:= ipSingleContour, hrPrev:= hr);
  // increment the iterator
  hr := F_VN_IncrementIterator(ipIterator:= ipIterator, hrPrev:= hr);

  // filter the contour on area size
  hr := F_VN_ContourArea(ipContour:= ipSingleContour, fArea:= fArea, hrPrev:= hr);
  IF fArea > 100_000 THEN
  ;
END_IF
```

11. Inside the above if statement we only have the contours of the blue parts such as was the intent of this sample project. Now we can focus on drawing the contours on the result image. To draw objects and to put text on images Beckhoff provides a whole list of drawing functions. In this exercise we will use functions such as

- **F_VN_DrawContours** | **F_VN_DrawContoursExp**
- **F_VN_DrawPoint** | **F_VN_DrawPointExp**
- **F_VN_DrawRotatedRectangle**
- **F_VN_PutText** | **F_VN_PutTextExp**

These functions all have a color argument to determine the color of the object or text. For this, we prepared some color variables in the declaration of MAIN_VISION

```

20      // colors
21      aColorRed      : TcVnVector4_LREAL := [255, 0, 0];
22      aColorGreen    : TcVnVector4_LREAL := [0, 255, 0];
23      aColorBlue     : TcVnVector4_LREAL := [0, 0, 255];
24      aColorBlack    : TcVnVector4_LREAL := [0, 0, 0];
25      aColorWhite    : TcVnVector4_LREAL := [255, 255, 255];
26      aColorYellow   : TcVnVector4_LREAL := [255, 255, 0];

```

Drawing of objects and text is typically done on the result image, *ipImageResult*.

12. Draw the contours of the blue parts with **F_VN_DrawContours**. *nContourIndex* should be -1, this means 'draw all contours in *ipContours*'.

```

IF fArea > 100_000 THEN
  // draw contours
  hr := F_VN_DrawContours(
    ipContours:= ipSingleContour,
    nContourIndex:= -1,
    ipDestImage:= ipImageResult,
    aColor:= aColorYellow,
    nThickness:= 5,
    hrPrev:= hr);
END_IF

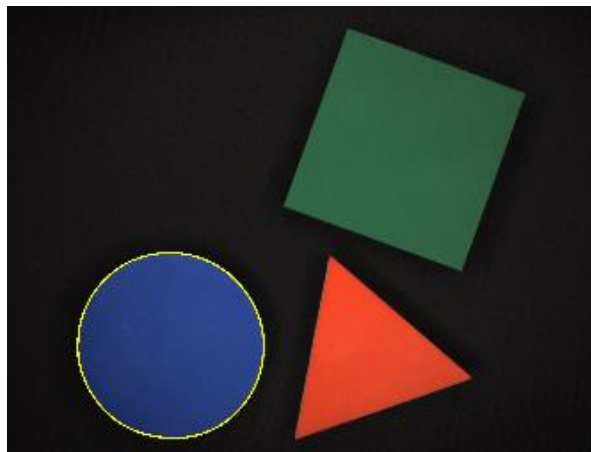
```

13. The code should look like the following up to now

```
// *****
// Add Vision Algorithms
// *****
hr := F_VN_CheckColorRange(
    ipSrcImage:= ipImageWork,
    ipDestImage:= ipImageWork,
    aLowerBounds:= aLowerBounds,
    aUpperBounds:= aUpperBounds,
    hrPrev:= hr);

// debug ==> Copy work Image to a displayable Work-Image (ipImageWork is still available after copy)
hr := F_VN_CopyIntoDisplayableImage(ipSrcImage:= ipImageWork, ipDestImage:= ipImageWorkDisp, hrPrev:= hr);
// Find contours
hr := F_VN_FindContours(ipSrcImage:= ipImageWork, ipContours:= ipContourList, hrPrev:= hr);
// create the iterator to loop through the elements
hr := F_VN_GetForwardIterator(ipContainer:= ipContourList, ipIterator:= ipIterator, hrPrev:= hr);
// iterate over the contour list
WHILE SUCCEEDED(hr) AND THEN ipIterator.CheckIfEnd() <> S_OK DO
    // extract a single contour out of the list
    hr := F_VN_GetContainer(ipIterator:= ipIterator, ipContainer:= ipSingleContour, hrPrev:= hr);
    // increment the iterator
    hr := F_VN_IncrementIterator(ipIterator:= ipIterator, hrPrev:= hr);
    // filter the contour on area size
    hr := F_VN_ContourArea(ipContour:= ipSingleContour, fArea:= fArea, hrPrev:= hr);
    IF fArea > 100_000 THEN
        // draw contours
        hr := F_VN_DrawContours(
            ipContours:= ipSingleContour,
            nContourIndex:= -1,
            ipDestImage:= ipImageResult,
            aColor:= aColorYellow,
            nThickness:= 5,
            hrPrev:= hr);
    END_IF
END_WHILE
```

Download and run the program up to this point. The result in the ADS Image Watch should look like the following picture. The following steps will add further objects and texts to the result image.



14. Determine the center of mass of the blue part with **F_VN_ContourCenterOfMass**. This function returns the X and Y coordinates (type **TcVnPoint2_LREAL**) of the center of mass. Afterwards, draw the center of mass on the result image. Create a variable, *fCenterOfMass*, of type **TcVnPoint2_LREAL** to store the center of mass coordinates:

```
fCenterOfMass : TcVnPoint2_LREAL;
```

Implement the functions continuing inside of the IF / END_IF construction

```
// Find center of mass
hr := F_VN_ContourCenterOfMass(
    ipContour:= ipSingleContour,
    aCenterOfMass:= fCenterOfMass,
    hrPrev:= hr);

// draw center of mass
hr := F_VN_DrawPointExp(
    nX:= TO_UDINT(fCenterOfMass[0]),
    nY:= TO_UDINT(fCenterOfMass[1]),
    ipDestImage:= ipImageResult,
    eShape:= ETcVnDrawShape.TCVN_DS_X,
    aColor:= aColorYellow,
    nSize:= 25,
    nThickness:= 5,
    eLineType:= ETcVnLineType.TCVN_LT_8_CONNECTED,
    hrPrev:= hr);
```

15. Determine the enclosing rectangle of the blue parts with **F_VN_EnclosingRectangle**. This function returns the enclosing rectangle (type **TcVnRotatedRectangle**) of the provided contour. This rectangle can be drawn with **F_VN_DrawRotatedRectangle**. Create a variable, *stRectangle*, to store the enclosing rectangle:

```
stRectangle : TcVnRotatedRectangle;
```

Implement the functions:

```
// Find enclosing rectangle
hr := F_VN_EnclosingRectangle(
    ipPointSet:= ipSingleContour,
    stRectangle:= stRectangle,
    hrPrev:= hr);

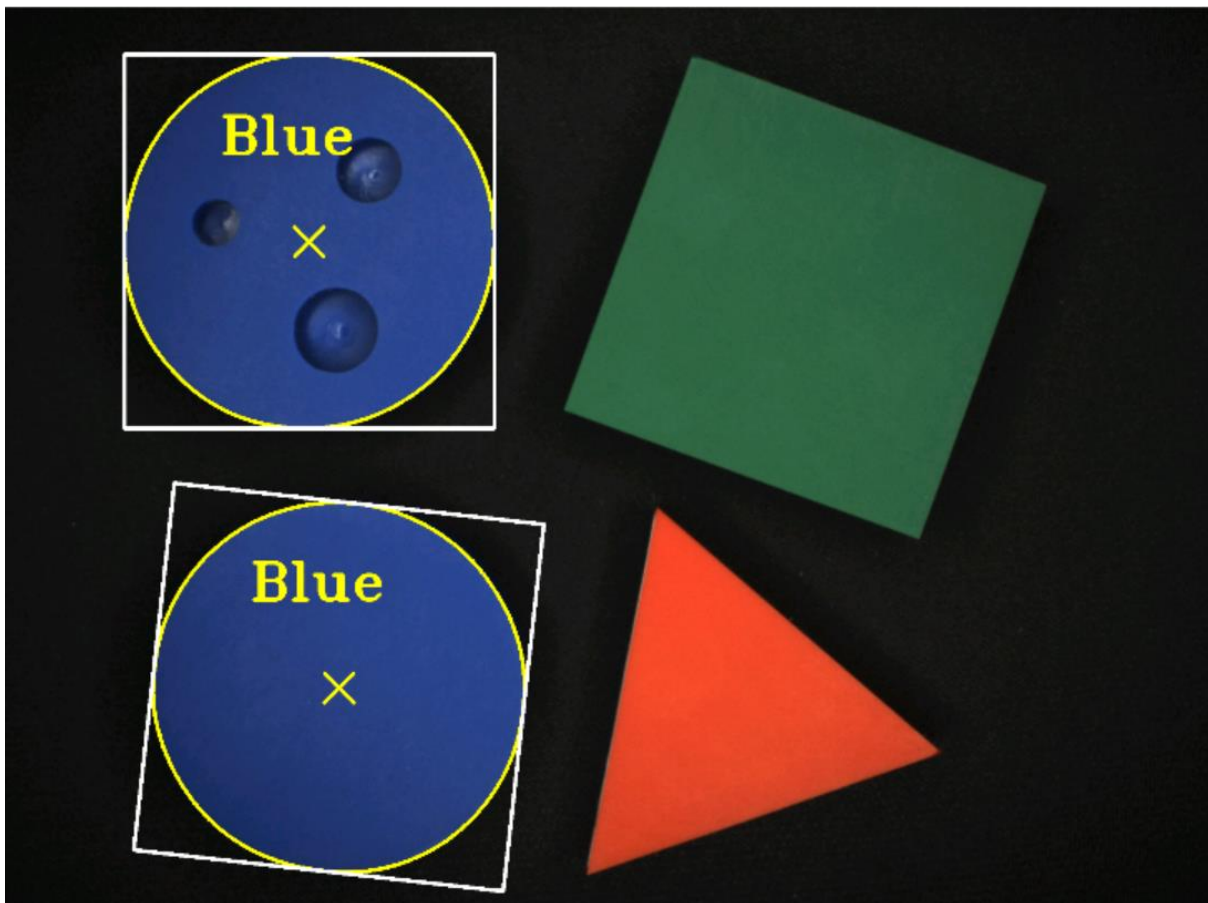
// Draw enclosing rectangle
hr := F_VN_DrawRotatedRectangle(
    stRectangle:= stRectangle,
    ipDestImage:= ipImageResult,
    aColor:= aColorWhite,
    nThickness:= 5,
    hrPrev:= hr);
```


16. As a last step, put the text 'Blue' next to each blue part on the result image with **F_VN_PutTextExp**.

```
// Put Text
hr := F_VN_PutTextExp(
    sText:= 'Blue',
    ipDestImage:= ipImageResult,
    nX:= TO_UDINT(fCenterOfMass[0]) - 150,
    nY:= TO_UDINT(fCenterOfMass[1]) - 150,
    eFontType:= ETcVnFontType.TCVN_FT_HERSHEY_TRIPLEX,
    fFontScale:= 3,
    aColor:= aColorYellow,
    nThickness:= 5,
    eLineType:= ETcVnLineType.TCVN_LT_8_CONNECTED,
    bBottomLeftOrigin:= FALSE,
    hrPrev:= hr);
```

17. Build the PLC project to check for errors. Login to the plc, implementing the latest additions

18. Proudly check the result of this sample project in the ADS Image Watch

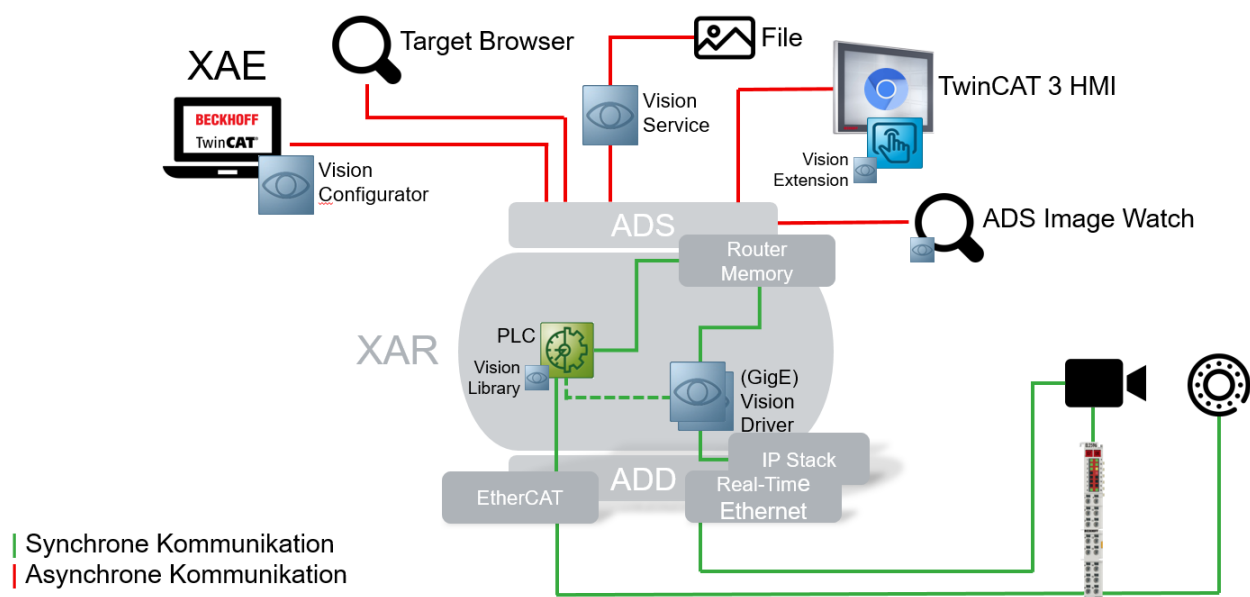


8. Project sample - TwinCAT Vision and TwinCAT HMI

8.1 Introduction

This exercise deals with the interaction of TwinCAT Vision and the TwinCAT HMI. At present, this is essentially a server extension that ensures that images from the PLC can be displayed in the TwinCAT HMI.

As explained in the introduction, the TwinCAT HMI Vision Extension is an extension for the TwinCAT HMI that enables transfer of images from the PLC to the TwinCAT HMI. Images from the PLC can be displayed directly in a web-based HMI with `tcHmiImageControl`.



All images are stored in the ADS router memory. With reference variable types *iTcVnImage* and *iTcVnDisplayableImage* references to these images in router memory are created so the PLC functions can interact with them. Remember that *iTcVnImage* images are used for processing, and *iTcVnDisplayableImage* images are made to be accessed over ADS and displayed e.g. in the ADS Image Watch.

The TwinCAT HMI Vision server extension enables the possibility to access and display the displayable images in the PLC directly on the HMI over ADS.

8.2 Project Sample

8.2.1 Objective

This exercise will cover:

- Adding the Vision HMI package to an HMI project.
- Displaying images directly on the HMI.

8.2.2 Commissioning

Since this chapter will not cover any PLC vision algorithms, we will use the finished exercise project from the chapter Image analysis: Colors and contours as a starting point.

The starting point for this sample is **Analysis.tszip**.

1. Open the tszip-archive located in the _Templates folder located in folder 03_Hmi and give your new solution a fitting name and directory.
2. Go to the Real-Time settings of the TwinCAT project and check the settings:
 - Make sure the Configured Size of the **Router Memory** is sufficiently large enough (go for minimum 256MB)
 - Read out the number of cores for your target and **configure the cores and tasks accordingly**. Ideally, use a minimum of two cores are available for TwinCAT, where one core is exclusively used for the VisionTask and the ImgAcq task.

The screenshot displays the TwinCAT configuration environment. On the left, the 'Solution Explorer' shows a project structure with folders like 'SYSTEM', 'PLC', and 'VISION'. The 'VISION' folder is expanded, showing 'VisionTask' and 'I/O' sub-folders. The main window shows the 'Settings' dialog for the 'Core Boost' tab. The 'Router Memory' is set to 512 MB, and 'Global Task Config' is set to 64KB. Below these settings are two tables:

Core	RT-Core	Base Time	Core Limit	Latency Warning	Core Memory	Core Frequency
0 (Sha...)	☐					Base Frequency
1 (Sha...)	☐					Base Frequency
2 (Sha...)	☒ Default	1 ms	80 %	(none)	512 KB with 2 KB limit	Base Frequency
3 (Isol...)	☒	1 ms	100 %	(none)	512 KB with 2 KB limit	Base Frequency

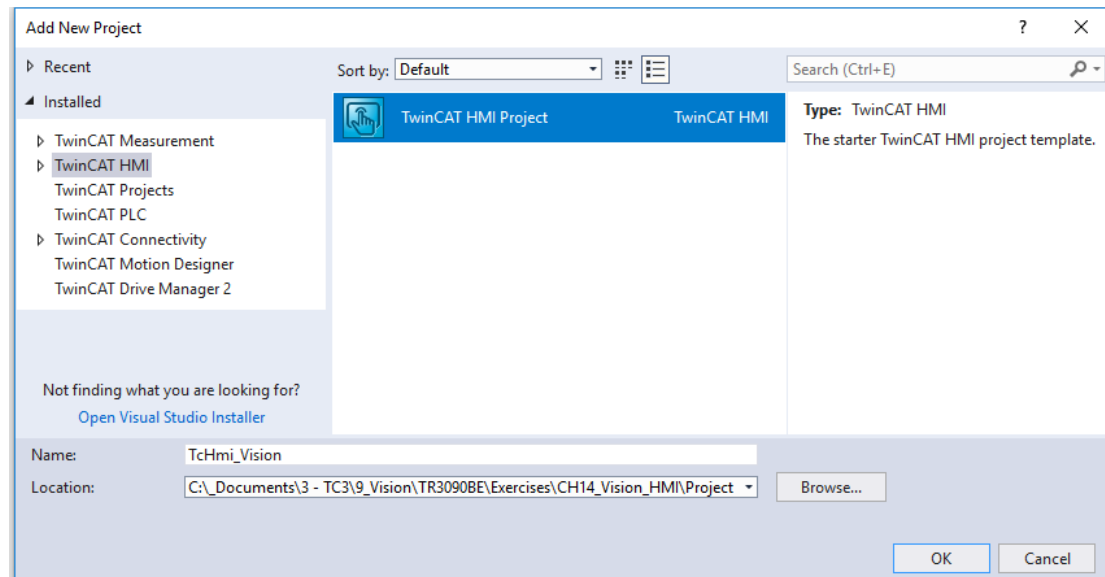
Object	RT-Core	Base Time (ms)	Cycle Time (ms)	Cycle Ticks	Priority
VisionTask	Core 3	1 ms	10 ms	10	1
VnTask-ImgAcq (FileSource1)	Core 3	1 ms	10 ms	10	5
I/O Idle Task	Default (2)	1 ms	1 ms	1	11
PicTask	Default (2)	1 ms	10 ms	10	20
AuxTask	Default (2)	1 ms	(none)	0	50

3. Activate the TwinCAT project and run/login to the PLC project. If you are experiencing a runtime crash immediately, then probably the router memory size is not set high enough.

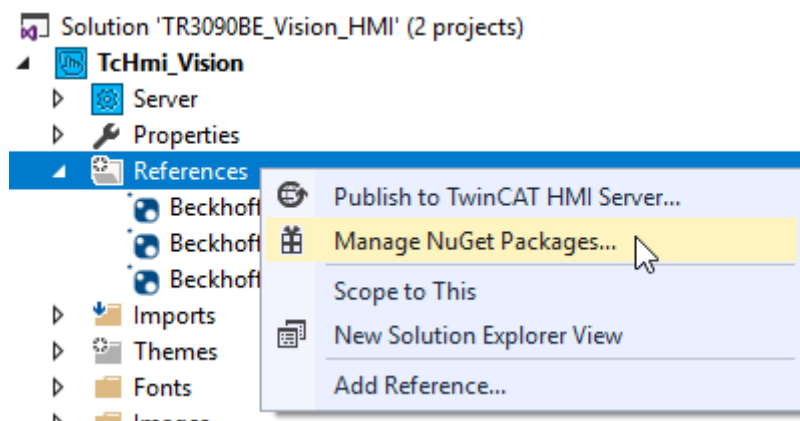
4. Open the **ADS Image Watch window** (View > Other Windows > ADS Image Watch) and select the correct target, choose port 851 and select **MAIN_Vision.iplImageDisp** to view the input images provided by FileSource1.

8.2.3 Implementation

1. Add a new HMI project to the solution and give it a fitting name.



2. Open the NuGet Package Manager for the HMI project by right clicking **References** and selecting **Manage NuGet Packages...**



3. TwinCAT 4026 ONLY (4024 below)

Install the necessary HMI controls for TwinCAT Vision via the TwinCAT Package manager command line. Therefor open a command line with administrative rights.

Execute 'TcPkg list vision' to list all vision packages. The HMI specific packages can be easily distinguished.

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.20348.3091]
(c) Microsoft Corporation. All rights reserved.

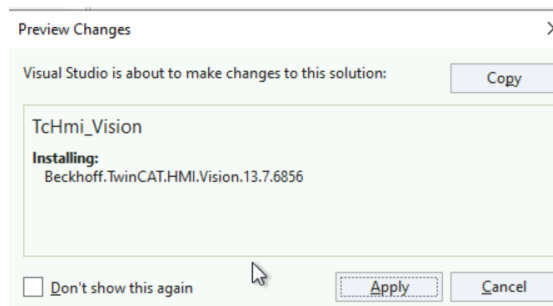
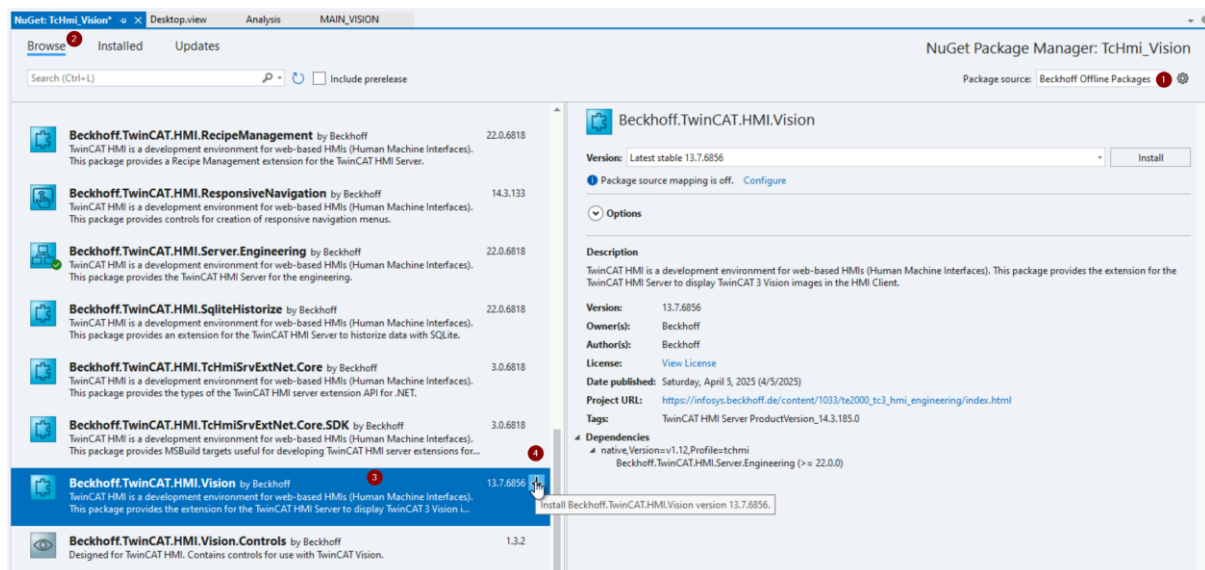
C:\Users\Administrator>tcpkg list vision
TcPkg 2.2.99

[Beckhoff Stable Feed] TF7xxx.Vision.XAE 5.7.4
[Beckhoff Stable Feed] TF7xxx.Vision.XAR 5.7.4
[Beckhoff Stable Feed] TwinCAT.HMI.Vision 14.3.185
[Beckhoff Stable Feed] TwinCAT.HMI.Vision.Controls 1.3.2
[Beckhoff Stable Feed] TwinCAT.XAE.GigEVision 5.7.4
[Beckhoff Stable Feed] TwinCAT.XAE.TMX.GigEVision 5.7.4
[Beckhoff Stable Feed] TwinCAT.XAE.TMX.Vision 5.7.4
[Beckhoff Stable Feed] TwinCAT.XAE.Vision 5.7.4
[Beckhoff Stable Feed] TwinCAT.XAE.VisionCommon 5.7.4
[Beckhoff Stable Feed] TwinCAT.XAR.VisionService 5.7.4
10 package(s) found.
```

Now install both packages by executing (for the exercise only pkg .Vision is needed)

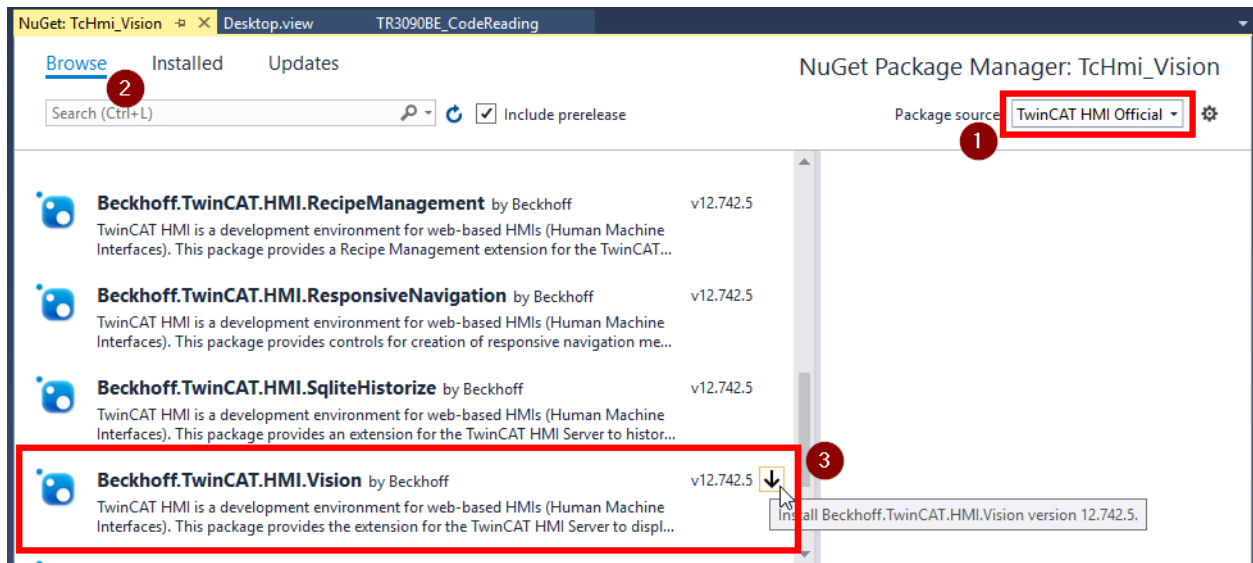
- tcpkg install TwinCAT.HMI.Vision
- tcpkg install TwinCAT.HMI.Vision.Controls

Set the package source to Beckhoff Offline Packages, go to 'Browse' (2) and scroll down to Beckhoff.TwinCAT.HMI.Vision. Click the install (arrow down) button (3) to install the Vision package.

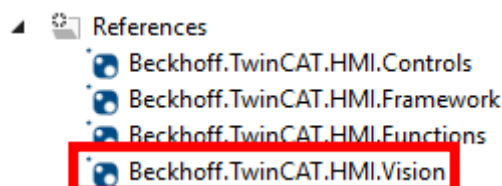


4. TwinCAT 4024 ONLY (Skip if using 4026)

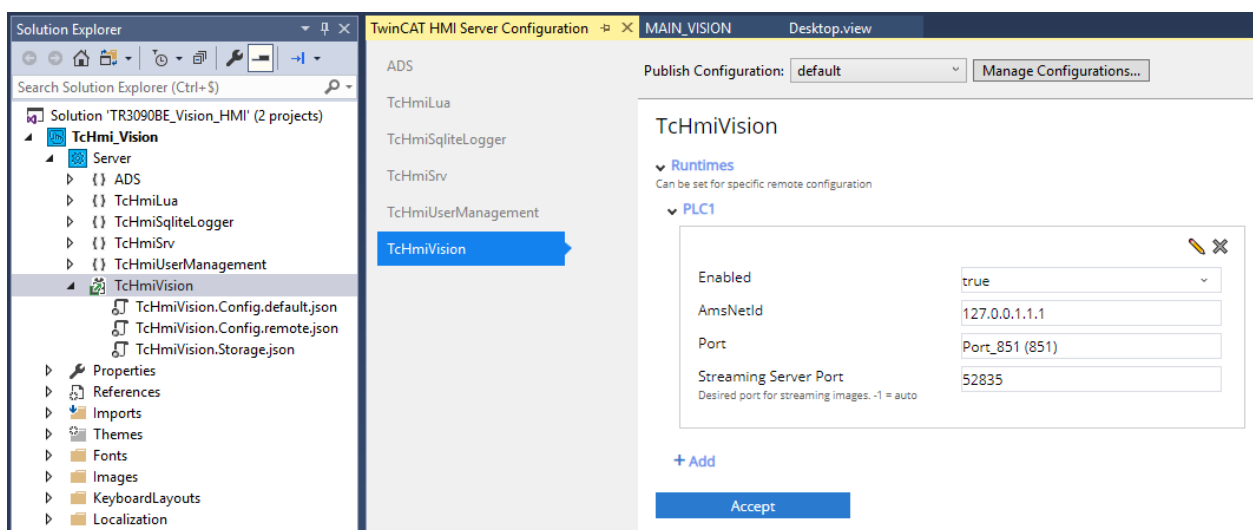
Set the package source to TwinCAT HMI Official, go to 'Browse' (2) and scroll down and select Beckhoff.TwinCAT.HMI.Vision(3). Click the install (arrow down) button (4) to install the Vision package.



5. The freshly installed vision package should now be available in the project's references.

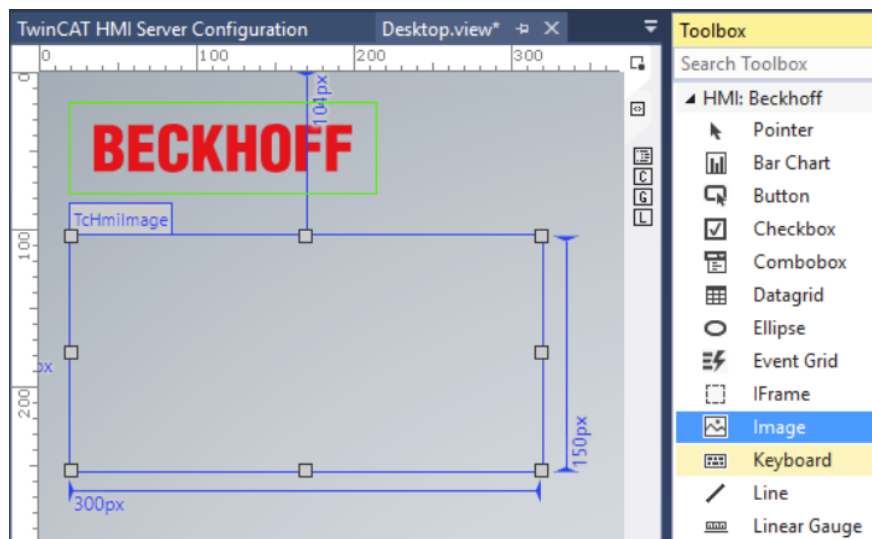


6. Once the package has been successfully integrated into the HMI project, enter the target PLC under **Server > Extensions > TcHmiVision**:

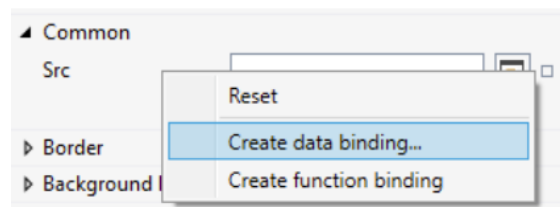


For the exercise it can stay as-is, pointing to the local runtime or it needs to be changed according to the settings of used target XAR (like a controller).

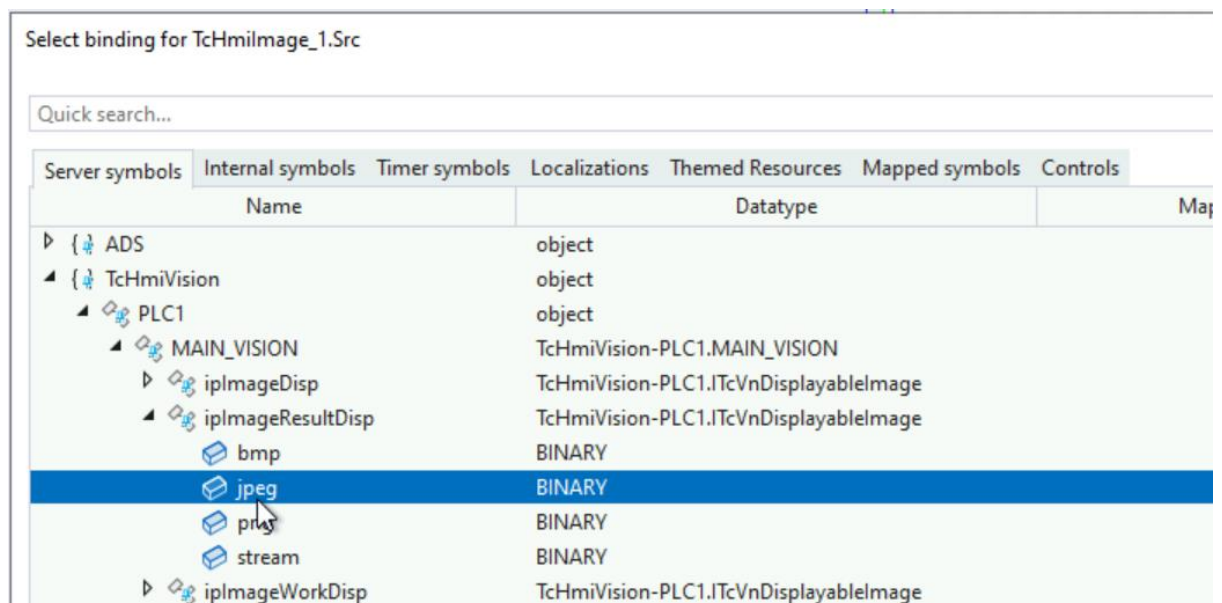
7. Go to the desktop view and add an Image control via the toolbox



- Open the properties of the control element and click on **Create data binding...** under the parameter **Src...**



9. Link the Src parameter of the image control with the corresponding icon as follows: **Server symbols > TcHmiVision > PLC1 > MAIN_Vision > iplImageResultDisp > < image format>**. We will use JPEG format, since this is fastest (but lossy). See the explanation of transmission types below.



Transmission formats

The server extension contains the image data from the PLC in BMP format and forwards them as a Base64 string to the TwinCAT HMI client. In order to reduce the data quantity transmitted to a client, images can be compressed in the server extension. The following image formats are available:

- **BMP**: uncompressed, ideal but possibly very large
- **JPEG**: compressed (lossy), good for natural images
- **PNG**: compressed (lossless), good for artificially created images
- **Stream**: For continuous image streams (e.g. live camera picture, not recommended!)

For transmission as a stream, the HMI server must be started as administrator.

Otherwise, no image transmission is possible.

10. Open the Live View and watch the selected TC Vision image being shown in the TC HMI.

